7 Data Types

Most programming languages include a notion of *type* for expressions and/or objects.[1] Types serve two principal purposes:

EXAMPLE 7.1
Operations that leverage type information

1. Types provide implicit context for many operations, so that the programmer does not have to specify that context explicitly. In C, for instance, the expression `a + b` will use integer addition if `a` and `b` are of `integer` type; it will use floating-point addition if `a` and `b` are of `double` (floating-point) type. Similarly, the operation `new p` in Pascal, where `p` is a pointer, will allocate a block of storage from the heap that is the right size to hold an object of the type pointed to by `p`; the programmer does not have to specify (or even know) this size. In C++, Java, and C#, the operation `new my_type()` not only allocates (and returns a pointer to) a block of storage sized for an object of type `my_type`; it also automatically calls any user-defined initialization (*constructor*) function that has been associated with that type. ■

EXAMPLE 7.2
Errors captured by type information

2. Types limit the set of operations that may be performed in a semantically valid program. They prevent the programmer from adding a character and a record, for example, or from taking the arctangent of a set, or passing a file as a parameter to a subroutine that expects an integer. While no type system can promise to catch every nonsensical operation that a programmer might put into a program by mistake, good type systems catch enough mistakes to be highly valuable in practice. ■

Section 7.1 looks more closely at the meaning and purpose of types, and presents some basic definitions. Section 7.2 addresses questions of *type equivalence* and *type compatibility*: when can we say that two types are the same, and when can we use a value of a given type in a given context? Sections 7.3–7.9 consider syntactic, semantic, and pragmatic issues for some of the most important

1 Recall that unless otherwise noted we are using the term "object" informally to refer to anything that might have a name. Object-oriented languages, which we will study in Chapter 9, assign a narrower, more formal, meaning to the term.

Programming Language Pragmatics. DOI: 10.1016/B978-0-12-374514-9.00017-3

composite types: records, arrays, strings, sets, pointers, lists, and files. The section on pointers includes a more detailed discussion of the value and reference models of variables introduced in Section 6.1.2, and of the heap management issues introduced in Section 3.2. The section on files (mostly on the PLP CD) includes a discussion of input and output mechanisms. Section 7.10 considers what it means to compare two complex objects for equality, or to assign one into the other.

7.1 Type Systems

Computer hardware can interpret bits in memory in several different ways: as instructions, addresses, characters, and integer and floating-point numbers of various lengths (see Section 5.2 for more details). The bits themselves, however, are untyped; the hardware on most machines makes no attempt to keep track of which interpretations correspond to which locations in memory. Assembly languages reflect this lack of typing: operations of any kind can be applied to values in arbitrary locations. High-level languages, on the other hand, almost always associate types with values, to provide the contextual information and error checking alluded to above.

Informally, a *type system* consists of (1) a mechanism to define types and associate them with certain language constructs, and (2) a set of rules for *type equivalence*, *type compatibility*, and *type inference*. The constructs that must have types are precisely those that have values, or that can refer to objects that have values. These constructs include named constants, variables, record fields, parameters, and sometimes subroutines; literal constants (e.g., `17`, `3.14`, `"foo"`); and more complicated expressions containing these. Type equivalence rules determine when the types of two values are the same. Type compatibility rules determine when a value of a given type can be used in a given context. Type inference rules define the type of an expression based on the types of its constituent parts or (sometimes) the surrounding context. In a language with polymorphic variables or parameters, it may be important to distinguish between the type of a reference or pointer and the type of the object to which it refers: a given name may refer to objects of different types at different times.

Subroutines are considered to have types in some languages, but not in others. Subroutines need to have types if they are first- or second-class values (i.e., if they can be passed as parameters, returned by functions, or stored in variables). In each of these cases there is a construct in the language whose value is a dynamically determined subroutine; type information allows the language to limit the set of acceptable values to those that provide a particular subroutine interface (i.e., particular numbers and types of parameters). In a statically scoped language that never creates references to subroutines dynamically (one in which subroutines are always third-class values), the compiler can always identify the subroutine to

which a name refers, and can ensure that the routine is called correctly without necessarily employing a formal notion of subroutine types.

7.1.1 Type Checking

Type checking is the process of ensuring that a program obeys the language's type compatibility rules. A violation of the rules is known as a *type clash*. A language is said to be *strongly typed* if it prohibits, in a way that the language implementation can enforce, the application of any operation to any object that is not intended to support that operation. A language is said to be *statically typed* if it is strongly typed and type checking can be performed at compile time. In the strictest sense of the term, few languages are statically typed. In practice, the term is often applied to languages in which most type checking can be performed at compile time, and the rest can be performed at run time.

A few examples: Ada is strongly typed, and for the most part statically typed (certain type constraints must be checked at run time). A Pascal implementation can also do most of its type checking at compile time, though the language is not quite strongly typed: untagged variant records (to be discussed in Section 7.3.4) are its only loophole. C89 is significantly more strongly typed than its predecessor dialects, but still significantly less strongly typed than Pascal. Its loopholes include unions, subroutines with variable numbers of parameters, and the interoperability of pointers and arrays (to be discussed in Section 7.7.1). Implementations of C rarely check anything at run time.

Dynamic (run-time) type checking is a form of late binding, and tends to be found in languages that delay other issues until run time as well. Lisp and Smalltalk are dynamically (though strongly) typed. Most scripting languages are also dynamically typed; some (e.g., Python and Ruby) are strongly typed. Languages with dynamic scoping are generally dynamically typed (or not typed at all): if the compiler can't identify the object to which a name refers, it usually can't determine the type of the object either.

7.1.2 Polymorphism

Polymorphism (Section 3.5.3) allows a single body of code to work with objects of multiple types. It may or may not imply the need for run-time type checking. As implemented in Lisp, Smalltalk, and the various scripting languages, fully dynamic typing allows the programmer to apply arbitrary operations to arbitrary objects. Only at run time does the language implementation check to see that the objects actually implement the requested operations. Because the types of objects can be thought of as implied (unspecified) parameters, dynamic typing is said to support *implicit parametric polymorphism*.

Unfortunately, while powerful and straightforward, dynamic typing incurs significant run-time cost, and delays the reporting of errors. ML and its descendants

employ a sophisticated system of *type inference* to support implicit parametric polymorphism in conjunction with static typing. The ML compiler infers for every object and expression a (possibly unique) type that captures precisely those properties that the object or expression must have to be used in the context(s) in which it appears. With rare exceptions, the programmer need not specify the types of objects explicitly. The task of the compiler is to determine whether there exists a consistent assignment of types to expressions that guarantees, statically, that no operation will ever be applied to a value of an inappropriate type at run time. This job can be formalized as the problem of *unification*; we discuss it further in Section 7.2.4.

In object-oriented languages, *subtype polymorphism* allows a variable X of type T to refer to an object of any type derived from T. Since derived types are required to support all of the operations of the base type, the compiler can be sure that any operation acceptable for an object of type T will be acceptable for any object referred to by X. Given a straightforward model of inheritance, type checking for subtype polymorphism can be implemented entirely at compile time. Most languages that envision such an implementation, including C++, Eiffel, Java, and C#, also provide *explicit parametric polymorphism (generics)*, which allow the programmer to define classes with type parameters. Generics are particularly useful for *container* (*collection*) classes: "list of `T`" (`List<T>`), "stack of `T`" (`Stack<T>`), and so on, where `T` is left unspecified. Like subtype polymorphism, generics can usually be type checked at compile time, though Java sometimes performs redundant checks at run time for the sake of interoperability with preexisting nongeneric code. Smalltalk, Objective-C, Python, and Ruby use a single mechanism for both parametric and subtype polymorphism, with

DESIGN & IMPLEMENTATION

Dynamic typing

The growing popularity of scripting languages has led a number of prominent software developers to publicly question the value of static typing. They ask: given that we can't check everything at compile time, how much pain is it worth to check the things we can? As a general rule, it is easier to write type-correct code than to prove that we have done so, and static typing requires such proofs. As type systems become more complex (due to object orientation, generics, etc.), the complexity of static typing increases correspondingly. Anyone who has written extensively in Ada or C++ on the one hand, and in Python or Scheme on the other, cannot help but be struck at how much easier it is to write code, at least for modest-sized programs, without complex type declarations. Dynamic checking incurs some run-time overhead, of course, and may delay the discovery of bugs, but this is increasingly seen as insignificant in comparison to the potential increase in human productivity. The choice between static and dynamic typing promises to provide one of the most interesting language debates of the coming decade.

checking delayed until run time. We will consider generics further in Section 8.4, and derived types in Chapter 9.

7.1.3 The Meaning of "Type"

Some early high-level languages (e.g., Fortran 77, Algol 60, and Basic) provided a small, built-in, and nonextensible set of types. As we saw in Section 3.3.1, Fortran does not require variables to be declared; it incorporates default rules to determine the type of undefined variables based on the spelling of their names (Basic has similar rules). As noted in the previous subsection, some languages (ML, Miranda, Haskell) infer types automatically at compile time, while others (Lisp, Smalltalk, scripting languages) track them at run time. In most languages, however, users must explicitly declare the type of every object, together with the characteristics of every type that is not built in.

There are at least three ways to think about types, which we may call the *denotational*, *constructive*, and *abstraction-based* points of view. From the denotational point of view, a type is simply a set of values. A value has a given type if it belongs to the set; an object has a given type if its value is guaranteed to be in the set. From the constructive point of view, a type is either one of a small collection of *built-in* types (integer, character, Boolean, real, etc.; also called *primitive* or *predefined* types), or a *composite* type created by applying a type *constructor* (`record`, `array`, `set`, etc.) to one or more simpler types. (This use of the term "constructor" is unrelated to the initialization functions of object-oriented languages. It also differs in a more subtle way from the use of the term in ML.) From the abstraction-based point of view, a type is an *interface* consisting of a set of operations with well-defined and mutually consistent semantics. For most programmers (and language designers), types usually reflect a mixture of these viewpoints.

In denotational semantics (one of the leading ways to formalize the meaning of programs), a set of values is known as a *domain*. Types are domains, and the meaning of an expression is a value from the domain that represents the expression's type. Some domains—the integers, for example—are simple and familiar. Others can be quite complex. In fact, in denotational semantics *everything* has a type—even statements with side effects. The meaning of an assignment statement is a value from a domain whose elements are functions. Each function maps a *store*—a mapping from names to values that represents the current contents of memory—to another store, which represents the contents of memory after the assignment.

One of the nice things about the denotational view of types is that it allows us in many cases to describe user-defined composite types (records, arrays, etc.) in terms of mathematical operations on sets. We will allude to these operations again in Section 7.1.4. Because it is based on mathematical objects, the denotational view of types usually ignores such implementation issues as limited precision and word length. This limitation is less serious than it might at first appear: Checks for such errors as arithmetic overflow are usually implemented outside of the type system

of a language anyway. They result in a run-time error, but this error is not called a type clash.

When a programmer defines an enumerated type (e.g., `enum hue {red, green, blue}` in C), he or she certainly thinks of this type as a set of values. For most other varieties of user-defined type, however, one typically does not think in terms of sets of values. Rather, one usually thinks in terms of the way the type is built from simpler types, or in terms of its meaning or purpose. These ways of thinking reflect the constructive and abstraction-based points of view. The constructive point of view was pioneered by Algol W and Algol 68, and is characteristic of most languages designed in the 1970s and 1980s. The abstraction-based point of view was pioneered by Simula-67 and Smalltalk, and is characteristic of modern object-oriented languages. It can also be adopted as a matter of programming discipline in non–object-oriented languages. We will discuss the abstraction-based point of view in more detail in Chapter 9. The remainder of this chapter focuses on the constructive point of view.

7.1.4 Classification of Types

The terminology for types varies some from one language to another. This subsection presents definitions for the most common terms. Most languages provide built-in types similar to those supported in hardware by most processors: integers, characters, Booleans, and real (floating-point) numbers.

Booleans (sometimes called *logicals*) are typically implemented as single-byte quantities, with `1` representing `true` and `0` representing `false`. C is unusual in its lack of a Boolean type: where most languages would expect a Boolean value, C expects an integer; zero means `false`, and anything else means `true`. As noted in Section ©6.5.4, Icon replaces Booleans with a more general notion of *success* and *failure*.

Characters have traditionally been implemented as one-byte quantities as well, typically (but not always) using the ASCII encoding. More recent languages (e.g., Java and C#) use a two-byte representation designed to accommodate (the commonly used portion of) the *Unicode* character set. Unicode is an international standard designed to capture the characters of a wide variety of languages (see sidebar on page 295). The first 128 characters of Unicode (`\u0000` through `\u007f`) are identical to ASCII. C++ provides both regular and "wide" characters, though for wide characters both the encoding and the actual width are implementation dependent. Fortran 2003 supports four-byte Unicode characters.

Numeric Types

A few languages (e.g., C and Fortran) distinguish between different lengths of integers and real numbers; most do not, and leave the choice of precision to the implementation. Unfortunately, differences in precision across language implementations lead to a lack of portability: programs that run correctly on one system may produce run-time errors or erroneous results on another. Java and C# are

unusual in providing several lengths of numeric types, with a specified precision for each.

A few languages, including C, C++, C# and Modula-2, provide both signed and unsigned integers (Modula-2 calls unsigned integers *cardinals*). A few languages (e.g., Fortran, C99, Common Lisp, and Scheme) provide a built-in complex type, usually implemented as a pair of floating-point numbers that represent the real and imaginary Cartesian coordinates; other languages support these as a standard library class. A few languages (e.g., Scheme and Common Lisp) provide a built-in rational type, usually implemented as a pair of integers that represent the numerator and denominator. Most scripting languages support integers of arbitrary

DESIGN & IMPLEMENTATION

Multilingual character sets

The *ISO 10646* international standard defines a *Universal Character Set (UCS)* intended to include all characters of all known human languages. (It also sets aside a "private use area" for such artificial [constructed] languages as Klingon, Tengwar, and Cirth [Tolkein Elvish]. Allocation of this private space is coordinated by a volunteer organization known as the ConScript Unicode Registry.) All natural languages currently employ codes in the 16-bit *Basic Multilingual Plane (BMP)*: `0x0000` through `0xfffd`.

Unicode is an expanded version of ISO 10646, maintained by an international consortium of software manufacturers. In addition to mapping tables, it covers such topics as rendering algorithms, directionality of text, and sorting and comparison conventions.

While recent languages have moved toward 16- or 32-bit internal character representations, these cannot be used for external storage—text files—without causing severe problems with backward compatibility. To accommodate Unicode without breaking existing tools, Ken Thompson in 1992 proposed a multibyte "expanding" code known as *UTF-8* (UCS/Unicode Transformation Format, 8-bit), and codified as a formal annex (appendix) to ISO 10646. UTF-8 characters occupy a maximum of 6 bytes—3 if they lie in the BMP, and only 1 if they are ordinary ASCII. The trick is to observe that ASCII is a 7-bit code; in any legacy text file the most significant bit of every byte is 0. In UTF-8 a most significant bit of 1 indicates a multibyte character. Two-byte codes begin with the bits `110`. Three-byte codes begin with `1110`. Second and subsequent bytes of multibyte characters always begin with `10`.

On some systems one also finds files encoded in one of 10 variants of the older 8-bit ISO 8859 standard, but these are inconsistently rendered across platforms. On the web, non-ASCII characters are typically encoded with *numeric character references*, which bracket a Unicode value, written in decimal or hex, with an ampersand and a semicolon. The copyright symbol (©), for example, is `&#169;`. Many characters also have symbolic *entity names* (e.g., `&copy;`), but not all browsers support these.

precision; the implementation uses multiple words of memory where appropriate. Ada supports *fixed-point* types, which are represented internally by integers, but have an implied decimal point at a programmer-specified position among the digits. Several languages support *decimal* types that use a base-10 encoding to avoid round-off anomalies in financial and human-centered arithmetic (see sidebar at bottom of this page).

Integers, Booleans, and characters are all examples of *discrete* types (also called *ordinal* types): the domains to which they correspond are countable (they have a one-to-one correspondence with some subset of the integers), and have a well-defined notion of predecessor and successor for each element other than the first and the last. (In most implementations the number of possible integers is finite, but this is usually not reflected in the type system.) Two varieties of user-defined types, enumerations and subranges, are also discrete. Discrete, rational, real, and

DESIGN & IMPLEMENTATION

Decimal types

A few languages, notably Cobol and PL/I, provide a *decimal* type for fixed-point representation of integer quantities. These types were designed primarily to exploit the *binary-coded decimal (BCD)* integer format supported by many traditional CISC machines. BCD devotes one *nibble* (four bits—half a byte) to each decimal digit. Machines that support BCD in hardware can perform arithmetic directly on the BCD representation of a number, without converting it to and from binary form. This capability is particularly useful in business and financial applications, which treat their data as both numbers and character strings.

With the growth in on-line commerce, the past few years have seen renewed interest in decimal arithmetic. The latest version of the IEEE 754 floating-point standard, adopted in June 2008, includes decimal floating-point types in 32-, 64-, and 128-bit lengths. These represent both the mantissa (significant bits) and exponent in binary, but interpret the exponent as a power of 10, not a power of 2. At a given length, values of decimal type have greater precision but smaller range than binary floating-point values. They are ideal for financial calculations, because they capture decimal fractions precisely. Designers hope the new standard will displace existing incompatible decimal formats, not only in hardware but also in software libraries, thereby providing the same portability and predictability that the original 754 standard provided for binary floating-point.

C# includes a 128-bit `decimal` type that is compatible with the new standard. Specifically, a C# decimal variable includes 96 bits of precision, a sign, and a decimal *scaling factor* that can vary between 10^{-28} and 10^{28}. IBM, for which business and financial applications have always been an important market, has included a hardware implementation of the standard (64- and 128-bit widths) in its POWER6 processor chips.

complex types together constitute the *scalar* types. Scalar types are also sometimes called *simple* types.

Enumeration Types

Enumerations were introduced by Wirth in the design of Pascal. They facilitate the creation of readable programs, and allow the compiler to catch certain kinds of programming errors. An enumeration type consists of a set of named elements. In Pascal, one can write:

EXAMPLE 7.3
Enumerations in Pascal

```
type weekday = (sun, mon, tue, wed, thu, fri, sat);
```

The values of an enumeration type are ordered, so comparisons are generally valid (`mon < tue`), and there is usually a mechanism to determine the predecessor or successor of an enumeration value (in Pascal, `tomorrow := succ(today)`). The ordered nature of enumerations facilitates the writing of enumeration-controlled loops:

```
for today := mon to fri do begin ...
```

It also allows enumerations to be used to index arrays:

```
var daily_attendance : array [weekday] of integer;
```

EXAMPLE 7.4
Enumerations as constants

An alternative to enumerations, of course, is simply to declare a collection of constants:

```
const sun = 0; mon = 1; tue = 2; wed = 3; thu = 4; fri = 5; sat = 6;
```

In C, the difference between the two approaches is purely syntactic:

```
enum weekday {sun, mon, tue, wed, thu, fri, sat};
```

is essentially equivalent to

```
typedef int weekday;
const weekday sun = 0, mon = 1, tue = 2,
              wed = 3, thu = 4, fri = 5, sat = 6;
```

In Pascal and most of its descendants, however, the difference between an enumeration and a set of integer constants is much more significant: the enumeration is a full-fledged type, incompatible with integers. Using an integer or an enumeration value in a context expecting the other will result in a type clash error at compile time.

EXAMPLE 7.5
Converting to and from enumeration type

Values of an enumeration type are typically represented by small integers, usually a consecutive range of small integers starting at zero. In many languages these *ordinal values* are semantically significant, because built-in functions can be used

to convert an enumeration value to its ordinal value, and sometimes vice versa. In Ada, these conversions employ the *attributes* `pos` and `val`: `weekday'pos(mon) = 1` and `weekday'val(1) = mon`.

EXAMPLE 7.6
Distinguished values for enums

Several languages allow the programmer to specify the ordinal values of enumeration types, if the default assignment is undesirable. In C, C++, and C#, one could write

```
enum mips_special_regs {gp = 28, fp = 30, sp = 29, ra = 31};
```

(The intuition behind these values is explained in Section 5.4.5.)

In Ada this declaration would be written

```
type mips_special_regs is (gp, sp, fp, ra);      -- must be sorted
for mips_special_regs use (gp => 28, sp => 29, fp => 30, ra => 31);
```

EXAMPLE 7.7
Emulating distinguished enum values in Java 5

In recent versions of Java one can obtain a similar effect by giving values an extra field (here named `register`):

```
enum mips_special_regs { gp(28), fp(30), sp(29), ra(31);
    private final int register;
    mips_special_regs(int r) { register = r; }
    public int reg() { return register; }
}
...
int n = mips_special_regs.fp.reg();
```

As noted in Section 3.5.2, Pascal and C do not allow the same element name to be used in more than one enumeration type in the same scope. Java and C# do, but the programmer must identify elements using fully qualified names: `mips_special_regs.fp`. Ada relaxes this requirement by saying that element names are overloaded; the type prefix can be omitted whenever the compiler can infer it from context.

Subrange Types

Like enumerations, subranges were first introduced in Pascal, and are found in many subsequent languages. A subrange is a type whose values compose a contiguous subset of the values of some discrete *base* type (also called the *parent* type). In Pascal and most of its descendants, one can declare subranges of integers, characters, enumerations, and even other subranges. In Pascal, subranges look like this:

EXAMPLE 7.8
Subranges in Pascal

```
type test_score = 0..100;
     workday = mon..fri;
```

EXAMPLE 7.9
Subranges in Ada

In Ada one would write

```
type test_score is new integer range 0..100;
subtype workday is weekday range mon..fri;
```

The `range...` portion of the definition in Ada is called a type *constraint*. In this example `test_score` is a *derived* type, incompatible with integers. The `workday` type, on the other hand, is a *constrained subtype*; `workday`s and `weekday`s can be more or less freely intermixed. The distinction between derived types and subtypes is a valuable feature of Ada; we will discuss it further in Section 7.2.1. ■

One could of course use integers to represent test scores, or a `weekday` to represent a `workday`. Using an explicit subrange has several advantages. For one thing, it helps to document the program. A comment could also serve as documentation, but comments have a bad habit of growing out of date as programs change, or of being omitted in the first place. Because the compiler analyzes a subrange declaration, it knows the expected range of subrange values, and can generate code to perform dynamic semantic checks to ensure that no subrange variable is ever assigned an invalid value. These checks can be valuable debugging tools. In addition, since the compiler knows the number of values in the subrange, it can sometimes use fewer bits to represent subrange values than it would need to use to represent arbitrary integers. In the example above, `test_score` values can be stored in a single byte.

EXAMPLE 7.10
Space requirements of subrange type

Most implementations employ the same bit patterns for integers and subranges, so subranges whose values are large require large storage locations, even if the number of distinct values is small. The following type, for example,

```
type water_temperature = 273..373;  (* degrees Kelvin *)
```

would be stored in at least two bytes. While there are only 101 distinct values in the type, the largest (373) is too large to fit in a single byte in its natural encoding. (An unsigned byte can hold values in the range 0..255; a signed byte can hold values in the range −128..127.) ■

DESIGN & IMPLEMENTATION

Multiple sizes of integers

The space savings possible with (small-valued) subrange types in Pascal and Ada is achieved in several other languages by providing more than one size of built-in integer type. C and C++, for example, support integer arithmetic on signed and unsigned variants of `char`, `short`, `int`, `long`, and (in C99) `long long` types, with monotonically nondecreasing sizes.[2]

2 More specifically, the C99 standard requires ranges for these types corresponding to lengths of at least 1, 2, 2, 4, and 8 bytes, respectively. In practice, one finds implementations in which plain `int`s are 2, 4, or 8 bytes long, including some in which they are the same size as `short`s but shorter than `long`s, and some in which they are the same size as `long`s, and longer than `short`s.

Composite Types

Nonscalar types are usually called *composite*, or *constructed* types. They are generally created by applying a *type constructor* to one or more simpler types. Common composite types include records (structures), variant records (unions), arrays, sets, pointers, lists, and files. All but pointers and lists are easily described in terms of mathematical set operations (pointers and lists can be described mathematically as well, but the description is less intuitive).

Records (structures) were introduced by Cobol, and have been supported by most languages since the 1960s. A record consists of collection of *fields*, each of which belongs to a (potentially different) simpler type. Records are akin to mathematical *tuples*; a record type corresponds to the Cartesian product of the types of the fields.

Variant records (unions) differ from "normal" records in that only one of a variant record's fields (or collections of fields) is valid at any given time. A variant record type is the *union* of its field types, rather than their Cartesian product.

Arrays are the most commonly used composite types. An array can be thought of as a function that maps members of an *index* type to members of a *component* type. Arrays of characters are often referred to as *strings*, and are often supported by special-purpose operations not available for other arrays.

Sets, like enumerations and subranges, were introduced by Pascal. A set type is the mathematical powerset of its base type, which must often be discrete. A variable of a set type contains a collection of distinct elements of the base type.

Pointers are l-values. A pointer value is a *reference* to an object of the pointer's base type. Pointers are often but not always implemented as addresses. They are most often used to implement *recursive* data types. A type T is recursive if an object of type T may contain one or more references to other objects of type T.

Lists, like arrays, contain a sequence of elements, but there is no notion of mapping or indexing. Rather, a list is defined recursively as either an empty list or a pair consisting of a head element and a reference to a sublist. While the length of an array must be specified at elaboration time in most (though not all) languages, lists are always of variable length. To find a given element of a list, a program must examine all previous elements, recursively or iteratively, starting at the head. Because of their recursive definition, lists are fundamental to programming in most functional languages.

Files are intended to represent data on mass-storage devices, outside the memory in which other program objects reside. Like arrays, most files can be conceptualized as a function that maps members of an index type (generally integer) to members of a component type. Unlike arrays, files usually have a notion of *current position*, which allows the index to be implied implicitly in consecutive operations. Files often display idiosyncrasies inherited from

physical input/output devices. In particular, the elements of some files must be accessed in sequential order.

We will examine composite types in more detail in Sections 7.3 through 7.9.

7.1.5 Orthogonality

In Section 6.1.2 we discussed the importance of orthogonality in the design of expressions, statements, and control-flow constructs. In a highly orthogonal language, these features can be used, with consistent behavior, in almost any combination. Orthogonality is equally important in type system design. A highly orthogonal language tends to be easier to understand, to use, and to reason about in a formal way. We have noted that languages like Algol 68 and C enhance orthogonality by eliminating (or at least blurring) the distinction between statements and expressions. To characterize a statement that is executed for its side effect(s), and that has no useful values, some languages provide an "empty" type. In C and Algol 68, for example, a subroutine that is meant to be used as a procedure is generally declared with a "return" type of `void`. In ML, the empty type is called `unit`. If the programmer wishes to call a subroutine that does return a value, but the value is not needed in this particular case (all that matters is the side effect[s]), then the return value in C can be discarded by "casting" it to `void` (casts will be discussed in Section 7.2.1):

EXAMPLE 7.11
`Void` (empty) type

```
foo_index = insert_in_symbol_table(foo);
...
(void) insert_in_symbol_table(bar);    /* don't care where it went */
    /* cast is optional; implied if omitted */
```

In a language (e.g., Pascal) without an empty type, the latter of these two calls would need to use a dummy variable:

EXAMPLE 7.12
Making do without `void`

```
var dummy : symbol_table_index;
...
dummy := insert_in_symbol_table(bar);
```

The type systems of C and Pascal are more orthogonal than that of (pre-Fortran 90) Fortran. Where array elements in traditional Fortran were always of scalar type, C and Pascal allow arbitrary types. Where array indices were always integers (and still are in C), Pascal allows any discrete type. Where function return values were always scalars (and still are in Pascal), C allows structures and pointers to functions. At the same time, both C and Pascal retain significant nonorthogonality. As in traditional Fortran, Pascal requires the bounds of each array to be specified at compile time, except when the array is a formal parameter of a subroutine. C requires a lower bound of zero on all array indices. Pascal has only second-class functions. A much more uniformly orthogonal type system can be found in ML; we consider it in Section 7.2.4

One particularly useful aspect of type orthogonality is the ability to specify literal values of arbitrary composite types. Composite literals are sometimes known as *aggregates*. They are particularly valuable for the initialization of static data structures; without them, a program may need to waste time performing initialization at run time.

EXAMPLE 7.13
Aggregates in Ada

Ada provides aggregates for all its structured types. Given the following declarations

```
type person is record
         name : string (1..10);
         age : integer;
     end record;
p, q : person;
A, B : array (1..10) of integer;
```

we can write the following assignments.

```
p := ("Jane Doe  ", 37);
q := (age => 36, name => "John Doe  ");
A := (1, 0, 3, 0, 3, 0, 3, 0, 0, 0);
B := (1 => 1, 3 | 5 | 7 => 3, others => 0);
```

Here the aggregates assigned into `p` and `A` are *positional*; the aggregates assigned into `q` and `B` name their elements explicitly. The aggregate for `B` uses a shorthand notation to assign the same value (`3`) into array elements `3`, `5`, and `7`, and to assign a `0` into all unnamed fields. Several languages, including C, Fortran 90, and Lisp, provide similar capabilities. ML provides a very general facility for composite expressions, based on the use of *constructors* (discussed in Section ©7.2.4). ■

CHECK YOUR UNDERSTANDING

1. What purpose(s) do types serve in a programming language?
2. What does it mean for a language to be *strongly typed*? *Statically typed*? What prevents, say, C from being strongly typed?
3. Name two important programming languages that are strongly but dynamically typed.
4. What is a *type clash*?
5. Discuss the differences among the *denotational*, *constructive*, and *abstraction-based* views of types.
6. What is the difference between *discrete* and *scalar* types?
7. Give two examples of languages that lack a Boolean type. What do they use instead?
8. In what ways may an enumeration type be preferable to a collection of named constants? In what ways may a subrange type be preferable to its base type? In what ways may a string be preferable to an array of characters?

9. What does it mean for a set of language features (e.g., a type system) to be *orthogonal*?

10. What are *aggregates*?

7.2 Type Checking

In most statically typed languages, every definition of an object (constant, variable, subroutine, etc.) must specify the object's type. Moreover, many of the contexts in which an object might appear are also typed, in the sense that the rules of the language constrain the types that an object in that context may validly possess. In the subsections below we will consider the topics of *type equivalence*, *type compatibility*, and *type inference*. Of the three, type compatibility is the one of most concern to programmers. It determines when an object of a certain type can be used in a certain context. At a minimum, the object can be used if its type and the type expected by the context are equivalent (i.e., the same). In many languages, however, compatibility is a looser relationship than equivalence: objects and contexts are often compatible even when their types are different. Our discussion of type compatibility will touch on the subjects of type *conversion* (also called *casting*), which changes a value of one type into a value of another; type *coercion*, which performs a conversion automatically in certain contexts; and *nonconverting* type casts, which are sometimes used in systems programming to interpret the bits of a value of one type as if they represented a value of some other type.

Whenever an expression is constructed from simpler subexpressions, the question arises: given the types of the subexpressions (and possibly the type expected by the surrounding context), what is the type of the expression as a whole? This question is answered by type inference. Type inference is often trivial: the sum of two integers is still an integer, for example. In other cases (e.g., when dealing with sets) it is a good bit trickier. Type inference plays a particularly important role in ML, Miranda, and Haskell, in which all type information is inferred.

7.2.1 Type Equivalence

In a language in which the user can define new types, there are two principal ways of defining type equivalence. *Structural equivalence* is based on the *content* of type definitions: roughly speaking, two types are the same if they consist of the same components, put together in the same way. *Name equivalence* is based on the *lexical occurrence* of type definitions: roughly speaking, each definition introduces a new type. Structural equivalence is used in Algol-68, Modula-3, and (with various wrinkles) C and ML. Name equivalence is the more popular approach in recent languages. It is used in Java, C#, standard Pascal, and most Pascal descendants, including Ada.

The exact definition of structural equivalence varies from one language to another. It requires that one decide which potential differences between types are important, and which may be considered unimportant. Most people would probably agree that the format of a declaration should not matter—otherwise identical declarations that differ only in spacing or line breaks should still be considered equivalent. Likewise, in a Pascal-like language with structural equivalence,

EXAMPLE 7.14
Trivial differences in type

```
type R2 = record
    a, b : integer
end;
```

should probably be considered the same as

```
type R3 = record
    a : integer;
    b : integer
end;
```

But what about

```
type R4 = record
    b : integer;
    a : integer
end;
```

Should the reversal of the order of the fields change the type? ML says no; most languages say yes. ■

EXAMPLE 7.15
Other minor differences in type

In a similar vein, consider the following arrays, again in a Pascal-like notation:

```
type str = array [1..10] of char;

type str = array [0..9] of char;
```

Here the length of the array is the same in both cases, but the index values are different. Should these be considered equivalent? Most languages say no, but some (including Fortran and Ada) consider them compatible. ■

To determine if two types are structurally equivalent, a compiler can expand their definitions by replacing any embedded type names with their respective definitions, recursively, until nothing is left but a long string of type constructors, field names, and built-in types. If these expanded strings are the same, then the types are equivalent, and conversely. Recursive and pointer-based types complicate matters, since their expansion does not terminate, but the problem is not insurmountable; we consider a solution in Exercise 7.19.

EXAMPLE 7.16
The problem with structural equivalence

Structural equivalence is a straightforward but somewhat low-level, implementation-oriented way to think about types. Its principal problem is an inability to distinguish between types that the programmer may think of as distinct, but which happen by coincidence to have the same internal structure:

```
1.  type student = record
2.          name, address : string
3.          age : integer

4.  type school = record
5.          name, address : string
6.          age : integer

7.  x : student;
8.  y : school;
9.  ...
10. x := y;                 -- is this an error?
```

Most programmers would probably want to be informed if they accidentally assigned a value of type school into a variable of type student, but a compiler whose type checking is based on structural equivalence will blithely accept such an assignment.

Name equivalence is based on the assumption that if the programmer goes to the effort of writing two type definitions, then those definitions are probably meant to represent different types. In the example above, variables x and y will be considered to have different types under name equivalence: x uses the type declared at line 1; y uses the type declared at line 4. ■

Variants of Name Equivalence

EXAMPLE 7.17
Alias types

One subtlety in the use of name equivalence arises in the simplest of type declarations:

```
TYPE new_type = old_type;           (* Modula-2 *)
```

Here `new_type` is said to be an *alias* for `old_type`. Should we treat them as two names for the same type, or as names for two different types that happen to have the same internal structure? The "right" approach may vary from one program to another. ■

EXAMPLE 7.18
Semantically equivalent alias types

In Example 3.13 we considered a module that needs to import a type name:

```
TYPE stack_element = INTEGER;       (* alias *)
MODULE stack;
IMPORT stack_element;
EXPORT push, pop;
    ...
PROCEDURE push(elem : stack_element);
    ...
PROCEDURE pop() : stack_element;
    ...
```

Here `stack` is meant to serve as an abstraction that allows the programmer, via textual inclusion, to create a stack of any desired type (in this case `INTEGER`). This code depends on alias types being considered equivalent; if they were not, the

programmer would have to replace `stack_element` with `INTEGER` everywhere it occurs.[3]

EXAMPLE 7.19
Semantically distinct alias types

Unfortunately, there are other times when aliased types should probably not be the same:

```
TYPE celsius_temp = REAL;
     fahrenheit_temp = REAL;
VAR  c : celsius_temp;
     f : fahrenheit_temp;
...
f := c;             (* this should probably be an error *)
```

A language in which aliased types are considered distinct is said to have *strict name equivalence*. A language in which aliased types are considered equivalent is said to have *loose name equivalence*. Most Pascal-family languages (including Modula-2) use loose name equivalence. Ada achieves the best of both worlds by allowing the programmer to indicate whether an alias represents a *derived* type or a *subtype*. A subtype is compatible with its base (parent) type; a derived type is incompatible. (Subtypes of the same base type are also compatible with each other.) Our examples above would be written:

EXAMPLE 7.20
Derived types and subtypes in Ada

```
subtype stack_element is integer;
...
type celsius_temp is new integer;
type fahrenheit_temp is new integer;
```

One way to think about the difference between strict and loose name equivalence is to remember the distinction between declarations and definitions (Section 3.3.3). Under strict name equivalence, a declaration `type A = B` is considered a definition. Under loose name equivalence it is merely a declaration; `A` shares the definition of `B`.

EXAMPLE 7.21
Name vs structural equivalence

Consider the following example:

```
1. type cell    = . . .          – – whatever
2. type alink   = pointer to cell
3. type blink   = alink
4. p, q : pointer to cell
5. r    : alink
6. s    : blink
7. t    : pointer to cell
8. u    : alink
```

3 One might argue here that the generics of more modern languages (Section 8.4) are a better way to build abstractions, but there are many single-use cases where generics would be overkill, and yet alias types still yield a more readable program.

Here the declaration at line 3 is an alias; it defines blink to be "the same as" alink. Under strict name equivalence, line 3 is both a declaration and a definition, and blink is a new type, distinct from alink. Under loose name equivalence, line 3 is just a declaration; it uses the definition at line 2.

Under strict name equivalence, p and q have the same type, because they both use the *anonymous* (unnamed) type definition on the right-hand side of line 4, and r and u have the same type, because they both use the definition at line 2. Under loose name equivalence, r, s, and u all have the same type, as do p and q. Under structural equivalence, all six of the variables shown have the same type, namely pointer to whatever cell is. ■

Both structural and name equivalence can be tricky to implement in the presence of separate compilation. We will return to this issue in Section 14.6.

Type Conversion and Casts

EXAMPLE 7.22

Contexts that expect a given type

In a language with static typing, there are many contexts in which values of a specific type are expected. In the statement

```
a := expression
```

we expect the right-hand side to have the same type as a. In the expression

```
a + b
```

the overloaded + symbol designates either integer or floating-point addition; we therefore expect either that a and b will both be integers, or that they will both be reals. In a call to a subroutine,

```
foo(arg1, arg2, . . . , argN)
```

we expect the types of the arguments to match those of the formal parameters, as declared in the subroutine's header. ■

Suppose for the moment that we require in each of these cases that the types (expected and provided) be exactly the same. Then if the programmer wishes to use a value of one type in a context that expects another, he or she will need to specify an explicit *type conversion* (also sometimes called a type *cast*). Depending on the types involved, the conversion may or may not require code to be executed at run time. There are three principal cases:

1. The types would be considered structurally equivalent, but the language uses name equivalence. In this case the types employ the same low-level representation, and have the same set of values. The conversion is therefore a purely conceptual operation; no code will need to be executed at run time.
2. The types have different sets of values, but the intersecting values are represented in the same way. One type may be a subrange of the other, for example, or one may consist of two's complement signed integers, while the other is

unsigned. If the provided type has some values that the expected type does not, then code must be executed at run time to ensure that the current value is among those that are valid in the expected type. If the check fails, then a dynamic semantic error results. If the check succeeds, then the underlying representation of the value can be used, unchanged. Some language implementations may allow the check to be disabled, resulting in faster but potentially unsafe code.

3. The types have different low-level representations, but we can nonetheless define some sort of correspondence among their values. A 32-bit integer, for example, can be converted to a double-precision IEEE floating-point number with no loss of precision. Most processors provide a machine instruction to effect this conversion. A floating-point number can be converted to an integer by rounding or truncating, but fractional digits will be lost, and the conversion will overflow for many exponent values. Again, most processors provide a machine instruction to effect this conversion. Conversions between different lengths of integers can be effected by discarding or sign-extending high-order bytes.

EXAMPLE 7.23
Type conversions in Ada

We can illustrate these options with the following examples of type conversions in Ada:

```
n : integer;          -- assume 32 bits
r : real;             -- assume IEEE double-precision
t : test_score;       -- as in Example 7.9
c : celsius_temp;     -- as in Example 7.20
...
t := test_score(n);     -- run-time semantic check required
n := integer(t);        -- no check req.; every test_score is an int
r := real(n);           -- requires run-time conversion
n := integer(r);        -- requires run-time conversion and check
n := integer(c);        -- no run-time code required
c := celsius_temp(n);   -- no run-time code required
```

In each of the six assignments, the name of a type is used as a pseudofunction that performs a type conversion. The first conversion requires a run-time check to ensure that the value of `n` is within the bounds of a `test_score`. The second conversion requires no code, since every possible value of `t` is acceptable for `n`. The third and fourth conversions require code to change the low-level representation of values. The fourth conversion also requires a semantic check. It is generally understood that converting from a floating-point value to an integer results in the loss of fractional digits; this loss is not an error. If the conversion results in integer overflow, however, an error needs to result. The final two conversions require no run-time code; the `integer` and `celsius_temp` types (at least as we have defined them) have the same sets of values and the same underlying representation. A purist might say that `celsius_temp` should be defined as `new integer range -273..integer'last`, in which case a run-time semantic check would be required on the final conversion. ■

EXAMPLE 7.24
Type conversions in C

A type conversion in C (what C calls a type cast) is specified by using the name of the desired type, in parentheses, as a prefix operator:

```
r = (float) n;  /* generates code for run-time conversion */
n = (int) r;    /* also run-time conversion, with no overflow check */
```

C and its descendants do not by default perform run-time checks for arithmetic overflow on any operation, though such checks can be enabled if desired in C#. ■

Nonconverting Type Casts Occasionally, particularly in systems programs, one needs to change the type of a value *without* changing the underlying implementation; in other words, to interpret the bits of a value of one type as if they were another type. One common example occurs in memory allocation algorithms, which use a large array of bytes to represent a heap, and then reinterpret portions of that array as pointers and integers (for bookkeeping purposes), or as various user-allocated data structures. Another common example occurs in high-performance numeric software, which may need to reinterpret a floating-point number as an integer or a record, in order to extract the exponent, significand, and sign fields. These fields can be used to implement special-purpose algorithms for square root, trigonometric functions, and so on.

EXAMPLE 7.25
Unchecked conversions in Ada

A change of type that does not alter the underlying bits is called a *nonconverting type cast*, or sometimes a *type pun*. It should not be confused with use of the term *cast* for conversions in languages like C. In Ada, nonconverting casts can be effected using instances of a built-in generic subroutine called `unchecked_conversion`:

```
-- assume 'float' has been declared to match IEEE single-precision
function cast_float_to_int is
    new unchecked_conversion(float, integer);
function cast_int_to_float is
    new unchecked_conversion(integer, float);
...
f := cast_int_to_float(n);
n := cast_float_to_int(f);
```

■

EXAMPLE 7.26
Conversions and nonconverting casts in C++

C++ inherits the casting mechanism of C, but also provides a family of semantically cleaner alternatives. Specifically, `static_cast` performs a type conversion, `reinterpret_cast` performs a nonconverting type cast, and `dynamic_cast` allows programs that manipulate pointers of polymorphic types to perform assignments whose validity cannot be guaranteed statically, but can be checked at run time (more on this in Chapter 9). Syntax for each of these is that of a generic function:

```
double d = ...
int n = static_cast<int>(d);
```

There is also a `const_cast` that can be used to remove read-only qualification. C-style type casts in C++ are defined in terms of `const_cast`, `static_cast`,

and `reinterpret_cast`; the precise behavior depends on the source and target types.

Any nonconverting type cast constitutes a dangerous subversion of the language's type system. In a language with a weak type system such subversions can be difficult to find. In a language with a strong type system, the use of explicit nonconverting type casts at least labels the dangerous points in the code, facilitating debugging if problems arise.

7.2.2 Type Compatibility

Most languages do not require equivalence of types in every context. Instead, they merely say that a value's type must be *compatible* with that of the context in which it appears. In an assignment statement, the type of the right-hand side must be compatible with that of the left-hand side. The types of the operands of + must both be compatible with some common type that supports addition (integers, real numbers, or perhaps strings or sets). In a subroutine call, the types of any arguments passed into the subroutine must be compatible with the types of the corresponding formal parameters, and the types of any formal parameters

DESIGN & IMPLEMENTATION

Nonconverting casts

C programmers sometimes attempt a nonconverting type cast (type pun) by taking the address of an object, converting the type of the resulting pointer, and then dereferencing:

```
r = *((float *) &n);
```

This arcane bit of hackery usually incurs no run-time cost, because most (but not all!) implementations use the same representation for pointers to integers and pointers to floating-point values—namely, an address. The ampersand operator (`&`) means "address of," or "pointer to." The parenthesized `(float *)` is the type name for "pointer to float" (float is a built-in floating-point type). The prefix `*` operator is a pointer dereference. The overall construct causes the compiler to interpret the bits of `n` as if it were a `float`. The reinterpretation will succeed only if `n` is an l-value (has an address), and `int`s and `float`s have the same size (again, this second condition is often but not always true in C). If `n` does not have an address then the compiler will announce a static semantic error. If `int` and `float` do not occupy the same number of bytes, then the effect of the cast may depend on a variety of factors, including the relative size of the objects, the alignment and "endian-ness" of memory (Section 5.2), and the choices the compiler has made regarding what to place in adjacent locations in memory. Safer and more portable nonconverting casts can be achieved in C by means of `union`s (variant records); we consider this option in Exercise 7.30.

passed back to the caller must be compatible with the types of the corresponding arguments.

The definition of type compatibility varies greatly from language to language. Ada takes a relatively restrictive approach: an Ada type `S` is compatible with an expected type `T` if and only if (1) `S` and `T` are equivalent, (2) one is a subtype of the other (or both are subtypes of the same base type), or (3) both are arrays, with the same numbers and types of elements in each dimension. Pascal is only slightly more lenient: in addition to allowing the intermixing of base and subrange types, it allows an integer to be used in a context where a real is expected.

Coercion

EXAMPLE 7.27
Coercion in Ada

Whenever a language allows a value of one type to be used in a context that expects another, the language implementation must perform an automatic, implicit conversion to the expected type. This conversion is called a *type coercion*. Like the explicit conversions discussed above, a coercion may require run-time code to perform a dynamic semantic check, or to convert between low-level representations. Ada coercions sometimes need the former, though never the latter:

```
d : weekday;          -- as in Example 7.3
k : workday;          -- as in Example 7.9
type calendar_column is new weekday;
c : calendar_column;
...
k := d;      -- run-time check required
d := k;      -- no check required; every workday is a weekday
c := d;      -- static semantic error;
             -- weekdays and calendar_columns are not compatible
```

To perform this third assignment in Ada we would have to use an explicit conversion:

```
c := calendar_column(d);
```

As we noted in Section 3.5.3, coercions are a controversial subject in language design. Because they allow types to be mixed without an explicit indication of intent on the part of the programmer, they represent a significant weakening of type security. C, which has a relatively weak type system, performs quite a bit of coercion. It allows values of most numeric types to be intermixed in expressions, and will coerce types back and forth "as necessary." Here are some examples:

EXAMPLE 7.28
Coercion in C

```
short int s;
unsigned long int l;
char c;     /* may be signed or unsigned -- implementation-dependent */
float f;    /* usually IEEE single-precision */
double d;   /* usually IEEE double-precision */
...
s = l;  /* l's low-order bits are interpreted as a signed number. */
l = s;  /* s is sign-extended to the longer length, then
           its bits are interpreted as an unsigned number. */
```

```
s = c;  /* c is either sign-extended or zero-extended to s's length;
           the result is then interpreted as a signed number. */
f = l;  /* l is converted to floating-point.  Since f has fewer
           significant bits, some precision may be lost. */
d = f;  /* f is converted to the longer format; no precision lost. */
f = d;  /* d is converted to the shorter format; precision may be lost.
           If d's value cannot be represented in single-precision, the
           result is undefined, but NOT a dynamic semantic error. */
```

Fortran 90 allows arrays and records to be intermixed if their types have the same *shape*. Two arrays have the same shape if they have the same number of dimensions, each dimension has the same size (i.e., the same number of elements), and the individual elements have the same shape. (In some other languages, the actual bounds of each dimension must be the same for the shapes to be considered the same.) Two records have the same shape if they have the same number of fields, and corresponding fields, in order, have the same shape. Field *names* do not matter, nor do the actual high and low bounds of array dimensions.

Ada's compatibility rules for arrays are roughly equivalent to those of Fortran 90. C provides no operations that take an entire array as an operand. C does, however, allow arrays and *pointers* to be intermixed in many cases; we will discuss this unusual form of type compatibility further in Section 7.7.1. Neither Ada nor C allows records (structures) to be intermixed unless their types are name equivalent.

In general, modern compiled languages display a trend toward static typing and away from type coercion. Some language designers have argued, however, that coercions are a natural way in which to support abstraction and program extensibility, by making it easier to use new types in conjunction with existing ones. This ease-of-programming argument is particularly important for scripting languages (Chapter 13). Among more traditional languages, C++ provides an extremely rich, *programmer-extensible* set of coercion rules. When defining a new type (a *class* in C++), the programmer can define coercion operations to convert values of the new type to and from existing types. These rules interact in complicated ways with the rules for resolving overloading (Section 3.5.2); they add significant flexibility to the language, but are one of the most difficult C++ features to understand and use correctly.

Overloading and Coercion

EXAMPLE 7.29
Coercion vs overloading of addends

We have noted (in Section 3.5.3) that overloading and coercion (as well as various forms of polymorphism) can sometimes be used to similar effect. It is worth repeating some of the distinctions here. An overloaded name can refer to more than one object; the ambiguity must be resolved by context. Consider the addition of numeric quantities. In the expression `a + b`, `+` may refer to either the integer or the floating-point addition operation. In a language without coercion, `a` and `b` must either both be integer or both be real; the compiler chooses the appropriate interpretation of `+` depending on their type. In a language with coercion, `+` refers

to the floating-point addition operation if either `a` or `b` is real; otherwise it refers to the integer addition operation. If only one of `a` and `b` is real, the other is coerced to match. One could imagine a language in which `+` was not overloaded, but rather referred to floating-point addition in all cases. Coercion could still allow `+` to take integer arguments, but they would always be converted to real. The problem with this approach is that conversions from integer to floating-point format take a non-negligible amount of time, especially on machines without hardware conversion instructions, and floating-point addition is significantly more expensive than integer addition.

In most languages literal constants (e.g., numbers, character strings, the empty set [`[ ]`] or the null pointer [`nil`]) can be intermixed in expressions with values of many types. One might say that constants are overloaded: `nil` for example might be thought of as referring to the null pointer value for whatever type is needed in the surrounding context. More commonly, however, constants are simply treated as a special case in the language's type-checking rules. Internally, the compiler considers a constant to have one of a small number of built-in "constant types" (int const, real const, string, null), which it then coerces to some more appropriate type as necessary, even if coercions are not supported elsewhere in the language. Ada formalizes this notion of "constant type" for numeric quantities: an integer constant (one without a decimal point) is said to have type `universal_integer`; a floating-point constant (one with an embedded decimal point and/or an exponent) is said to have type `universal_real`. The `universal_integer` type is compatible with any type derived from `integer`; `universal_real` is compatible with any type derived from `real`.

Universal Reference Types

For systems programming, or to facilitate the writing of general-purpose *container* (*collection*) objects (lists, stacks, queues, sets, etc.) that hold references to other objects, several languages provide a *universal* reference type. In C and C++, this type is called `void*`. In Clu it is called `any`; in Modula-2, `address`; in Modula-3, `refany`; in Java, `Object`; in C#, `object`. Arbitrary l-values can be assigned into an object of universal reference type, with no concern about type safety: because the type of the object referred to by a universal reference is unknown, the compiler will not allow any operations to be performed on that object. Assignments back into objects of a particular reference type (e.g., a pointer to a programmer-specified record type) are a bit trickier, if type safety is to be maintained. We would not want a universal reference to a floating-point number, for example, to be assigned into a variable that is supposed to hold a reference to an integer, because subsequent operations on the "integer" would interpret the bits of the object incorrectly. In object-oriented languages, the question of how to ensure the validity of a universal to specific assignment generalizes to the question of how to ensure the validity of any assignment in which the type of the object on left-hand side supports operations that the object on the right-hand side may not.

One way to ensure the safety of universal to specific assignments (or, in general, less specific to more specific assignments) is to make objects self-descriptive—that

is, to include in the representation of each object a *tag* that indicates its type. This approach is common in object-oriented languages, which generally need it for dynamic method binding. Type tags in objects can consume a nontrivial amount of space, but allow the implementation to prevent the assignment of an object of one type into a variable of another. In Java and C#, a universal to specific assignment requires a type cast, and will generate an exception if the universal reference does not refer to an object of the casted type. In Eiffel, the equivalent operation uses a special assignment operator (`?=` instead of `:=`); in C++ it uses a `dynamic_cast` operation.

EXAMPLE 7.30
Java container of `Object`

Java and C# programmers frequently create container classes that hold objects of the universal reference class (`Object` or `object`, respectively). When an object is removed from a container, it must be assigned (with a type cast) into a variable of an appropriate class before anything interesting can be done with it:[4]

```
import java.util.*;     // library containing Stack container class
...
Stack myStack = new Stack();
String s = "Hi, Mom";
Foo f = new Foo();      // f is of user-defined class type Foo
...
myStack.push(s);
myStack.push(f);        // we can push any kind of object on a stack
...
s = (String) myStack.pop();
    // type cast is required, and will generate an exception at run
    // time if element at top-of-stack is not a string
```

In a language without type tags, the assignment of a universal reference into an object of a specific reference type cannot be checked, because objects are not self-descriptive: there is no way to identify their type at run time. The programmer must therefore resort to an (unchecked) type conversion.

7.2.3 Type Inference

We have seen how type checking ensures that the components of an expression (e.g., the arguments of a binary operator) have appropriate types. But what determines the type of the overall expression? In most cases, the answer is easy. The result of an arithmetic operator usually has the same type as the operands. The result of a comparison is usually Boolean. The result of a function call has the type declared in the function's header. The result of an assignment (in languages in which assignments are expressions) has the same type as the left-hand side. In

4 If the programmer knows that a container will be used to hold objects of only one type, then it may be possible to eliminate the type cast and, ideally, its run-time cost by using generics (Section 8.4).

a few cases, however, the answer is not obvious. In particular, operations on subranges and on composite objects do not necessarily preserve the types of the operands. We examine these cases in the remainder of this subsection. We then consider (on the PLP CD) a more elaborate form of type inference found in ML, Miranda, and Haskell.

Subranges

EXAMPLE 7.31
Inference of subrange types

For simple arithmetic operators, the principal type system subtlety arises when one or more operands have subrange types (what Ada calls subtypes with range constraints). Given the following Pascal definitions, for example,

```
type Atype = 0..20;
     Btype = 10..20;
var  a : Atype;
     b : Btype;
```

what is the type of `a + b`? Certainly it is neither `Atype` nor `Btype`, since the possible values range from 10 to 40. One could imagine it being a new anonymous subrange type with 10 and 40 as bounds. The usual answer in Pascal and its descendants is to say that the result of any arithmetic operation on a subrange has the subrange's base type, in this case integer.

If the result of an arithmetic operation is assigned into a variable of a subrange type, then a dynamic semantic check may be required. To avoid the expense of some unnecessary checks, a compiler may keep track at compile time of the largest and smallest possible values of each expression, in essence computing the anonymous `10...40` type. More sophisticated techniques can be used to eliminate many checks in loops; we will consider these in Section 16.5.2. ■

EXAMPLE 7.32
The `var` placeholder in C# 3.0

In languages like Ada, the type of an arithmetic expression assumes special significance in the header of a `for` loop (Section 6.5.1), because it determines the type of the index variable. For the sake of uniformity, Ada says that the index of a `for` loop always has the base type of the loop bounds, whether they are built-up expressions or simple variables or constants. A similar convention appears in C# 3.0, which allows a variable declaration to use the placeholder `var` instead of a type name when an appropriate type can be inferred from the initialization expression:

```
var i = 123;                                  // equiv. to int i = 123;
var map = new Dictionary<int, string>();  // equiv. to
    // Dictionary<int, string> map = new Dictionary<int, string>();
```
■

Composite Types

Most built-in operators in most languages take operands of built-in types. Some operators, however, can be applied to values of composite types, including aggregates. Type inference becomes an issue when an operation on composites yields a result of a different type than the operands.

EXAMPLE 7.33
Type inference on string operations

Character strings provide a simple example. In Pascal, the literal string `'abc'` has type `array [1..3] of char`. In Ada, the analogous string (denoted `"abc"`) is considered to have an incompletely specified type that is compatible with any three-element array of characters. In the Ada expression `"abc" & "defg"`, `"abc"` is a three-character array, `"defg"` is a four-character array, and the result is a seven-character array formed by concatenating the two. For all three, the size of the array is known but the bounds and the index type are not; they must be inferred from context. The seven-character result of the concatenation could be assigned into an array of type `array (1..7) of character` or into an array of type `array (weekday) of character`, or into any other seven-element character array. ■

EXAMPLE 7.34
Type inference for sets

Operations on composite values also occur when manipulating sets. Pascal and Modula, for example, support union (`+`), intersection (`*`), and difference (`-`) on sets of discrete values. Set operands are said to have compatible types if their elements have the same base type `T`. The result of a set operation is then of type `set of T`. As with subranges, a compiler can avoid the need for run-time bounds checks in certain cases by keeping track of the minimum and maximum possible members of the set expression. Because a set may have many members, some of which may be known at compile time, it can be useful to track not only the largest and smallest values that may be in a set, but also the values that are known to be in the set (see Exercise 7.7). ■

7.2.4 The ML Type System

The most sophisticated form of type inference occurs in certain functional languages, notably ML, Miranda, and Haskell. Programmers have the option of declaring the types of objects in these languages, in which case the compiler behaves much like that of a more traditional statically typed language. As we noted near the beginning of Section 7.1, however, programmers may also choose not to declare certain types, in which case the compiler will infer them, based on the known types of literal constants, the explicitly declared types of any objects that have them, and the syntactic structure of the program. ML-style type inference is the invention of the language's creator, Robin Milner.[5]

IN MORE DEPTH

An introduction to the type system of ML and its descendants appears on the PLP CD. The key to its inference mechanism is to *unify* the (partial) type information

5 Robin Milner (1934–), of Cambridge University's Computer Laboratory, is responsible not only for the development of ML and its type system, but for the Logic of Computable Functions, which provides a formal basis for machine-assisted proof construction, and the Calculus of Communicating Systems, which provides a general theory of concurrency. He received the ACM Turing Award in 1991.

available for two expressions whenever the rules of the type system say that their types must be the same. Information known about each is then known about the other as well. Any discovered inconsistencies are identified as static semantic errors. Any expression whose type remains incompletely specified after inference is automatically polymorphic; this is the *implicit parametric polymorphism* referred to in Section 3.5.3. ML family languages also incorporate a powerful run-time pattern-matching facility, and several unconventional structured types, including ordered tuples, (unordered) records, lists, and a `datatype` mechanism that subsumes unions and recursive types.

✓ CHECK YOUR UNDERSTANDING

11. What is the difference between *type equivalence* and *type compatibility*?
12. Discuss the comparative advantages of *structural* and *name* equivalence for types. Name three languages that use each approach.
13. Explain the differences among *strict* and *loose* name equivalence.
14. Explain the distinction between *derived* types and *subtypes* in Ada.
15. Explain the differences among *type conversion*, *type coercion*, and *nonconverting type casts*.
16. Summarize the arguments for and against coercion.
17. Under what circumstances does a type conversion require a run-time check?
18. What purpose is served by *universal* reference types?
19. What is *type inference*? Describe three contexts in which it occurs.

7.3 Records (Structures) and Variants (Unions)

Record types allow related data of heterogeneous types to be stored and manipulated together. Some languages (notably Algol 68, C, C++, and Common Lisp) use the term *structure* (declared with the keyword `struct`) instead of *record*. Fortran 90 simply calls its records "types": they are the only form of programmer-defined type other than arrays, which have their own special syntax. Structures in C++ are defined as a special form of *class* (one in which members are globally visible by default). Java has no distinguished notion of `struct`; its programmers use classes in all cases. C# uses a reference model for variables of `class` types, and a value model for variables of `struct` types. C# `structs` do not support inheritance. For the sake of simplicity, we will use the term "record" in most of our discussion to refer to the relevant construct in all these languages.

7.3.1 Syntax and Operations

EXAMPLE 7.35
A C struct

In C, a simple record might be defined as follows.

```
struct element {
    char name[2];
    int atomic_number;
    double atomic_weight;
    _Bool metallic;
};
```

EXAMPLE 7.36
A Pascal record

In Pascal, the corresponding declarations would be

```
type two_chars = packed array [1..2] of char;
    (* Packed arrays will be explained in Example 7.43.
       Packed arrays of char are compatible with quoted strings. *)
type element = record
    name : two_chars;
    atomic_number : integer;
    atomic_weight : real;
    metallic : Boolean
end;
```

EXAMPLE 7.37
Accessing record fields

Each of the record components is known as a *field*. To refer to a given field of a record, most languages use "dot" notation. In C:

```
element copper;
const double AN = 6.022e23;     /* Avogadro's number */
...
copper.name[0] = 'C'; copper.name[1] = 'u';
double atoms = mass / copper.atomic_weight * AN;
```

Pascal notation is similar to that of C. In Fortran 90 one would say `copper%name` and `copper%atomic_weight`. Cobol and Algol 68 reverse the order of the field and record names: `name of copper` and `atomic_weight of copper`. ML's notation is

DESIGN & IMPLEMENTATION

Struct tags and `typedef` in C and C++

One of the peculiarities of the C type system is that `struct` tags are not exactly type names. In Example 7.35, the name of the type is the two-word phrase `struct element`. We used this name to declare the `element_yielded` field of the second struct in Example 7.38. To obtain a one-word name, one can say `typedef struct element element_t`, or even `typedef struct element element`: struct tags and `typedef` names have separate name spaces, so the same name can be used in each. C++ eliminates this idiosyncrasy by allowing the struct tag to be used as a type name without the `struct` prefix; in effect, it performs the `typedef` implicitly.

also "reversed," but uses a prefix `#`: `#name copper` and `#atomic_weight copper`. (Fields of an ML record can also be extracted using patterns.) In Common Lisp, one would say `(element-name copper)` and `(element-atomic_weight copper)`.

EXAMPLE 7.38
Nested records

Most languages allow record definitions to be nested. Again in C:

```
struct ore {
    char name[30];
    struct {
        char name[2];
        int atomic_number;
        double atomic_weight;
        _Bool metallic;
    } element_yielded;
};
```

Alternatively, one could say

```
struct ore {
    char name[30];
    struct element element_yielded;
};
```

In Fortran 90 and Common Lisp, only the second alternative is permitted: record fields can have record types, but the declarations cannot be lexically nested. Naming for nested records is straightforward: `malachite.element_yielded.atomic_number` in Pascal or C; `atomic_number of element_yielded of malachite` in Cobol; `#atomic_number #element_yielded malachite` in ML; `(element-atomic_number (ore-element_yielded malachite))` in Common Lisp.

EXAMPLE 7.39
ML records and tuples

As noted in Example 7.14, ML differs from most languages in specifying that the order of record fields is insignificant. The ML record value `{name = "Cu", atomic_number = 29, atomic_weight = 63.546, metallic = true}` is the same as the value `{atomic_number = 29, name = "Cu", atomic_weight = 63.546, metallic = true}` (they will test `true` for equality). ML tuples are defined as abbreviations for records whose field names are small integers. The values `("Cu", 29)`, `{1 = "Cu", 2 = 29}`, and `{2 = 29, 1 = "Cu"}` will all test `true` for equality.

7.3.2 Memory Layout and Its Impact

The fields of a record are usually stored in adjacent locations in memory. In its symbol table, the compiler keeps track of the offset of each field within each record type. When it needs to access a field, the compiler typically generates a load or store instruction with displacement addressing. For a local object, the base register is the frame pointer; the displacement is the sum of the record's offset from the register and the field's offset within the record. On a RISC machine, a global record

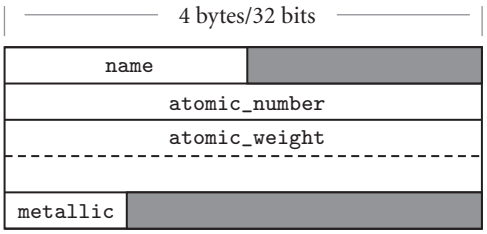


Figure 7.1 **Likely layout in memory for objects of type `element` on a 32-bit machine.** Alignment restrictions lead to the shaded "holes."

is accessed in a similar way, using a dedicated *globals pointer* register as base. On a CISC machine, the compiler may access the field directly at its absolute address or, if many fields are to be accessed in a short period of time, it may load a temporary register with the (absolute) address of the record and then use the field's offset as displacement.

EXAMPLE 7.40
Memory layout for a record type

A likely layout for our `element` type on a 32-bit machine appears in Figure 7.1. Because the `name` field is only two characters long, it occupies two bytes in memory. Since `atomic_number` is an integer, and must (on most machines) be word-aligned, there is a two-byte "hole" between the end of `name` and the beginning of `atomic_number`. Similarly, since Boolean variables (in most language implementations) occupy a single byte, there are three bytes of empty space between the end of the `metallic` field and the next aligned location. In an array of `elements`, most compilers would devote 20 bytes to every member of the array. ■

In a language with a value model of variables, nested records are naturally embedded in the parent record, where they function as large fields with word or double-word alignment. In a language with a reference model of variables, fields of record type are typically references to data in another location. The difference is a matter not only of memory layout, but also of semantics. In Pascal, the following program prints a 0.

EXAMPLE 7.41
Nested records as values

```
type
    T = record
        j : integer;
    end;
    S = record
        i : integer;
        n : T;
    end;
var s1, s2 : S;
...
s1.n.j := 0;
s2 := s1;
s2.n.j := 7;
writeln(s1.n.j);        (* prints 0 *)
```

The assignment of `s1` into `s2` copies the embedded `T`. ■

EXAMPLE 7.42
Nested records as references

By contrast, the following Java program prints a 7. (Simple classes in Java play the role of structs.)

```
class T {
    public int j;
}
class S {
    public int i;
    public T n;
}
...
S s1 = new S();
s1.n = new T();                 // fields initialized to 0
S s2 = s1;
s2.n.j = 7;
System.out.println(s1.n.j);     // prints 7
```

Here the assignment of `s1` into `s2` has copied only the reference, so `s2.n.j` is an alias for `s1.n.j`.

EXAMPLE 7.43
Layout of `packed` types

A few languages—notably Pascal—allow the programmer to specify that a record type (or an array, set, or file type) should be *packed*:

```
type element = packed record
    name : two_chars;
    atomic_number : integer;
    atomic_weight : real;
    metallic : Boolean
end;
```

The keyword `packed` indicates that the compiler should optimize for space instead of speed. In most implementations a compiler will implement a packed record without holes, by simply "pushing the fields together." To access a nonaligned field, however, it will have to issue a multi-instruction sequence that retrieves the pieces of the field from memory and then reassembles them in a register. A likely packed layout for our `element` type (again for a 32-bit machine) appears in Figure 7.2. It is 15 bytes in length. An array of packed `element` records would probably devote 16 bytes to each member of the array; that is, it would align each element. A packed array of packed records might devote only 15 bytes to each; only every fourth element would be aligned. Ada, Modula-3, and C provide more elaborate packing mechanisms, which allow the programmer to specify precisely how many bits are to be devoted to each field.

EXAMPLE 7.44
Assignment and comparison of records

Most languages allow a value to be assigned to an entire record in a single operation:

```
my_element := copper;
```

Ada also allows records to be compared for equality (`if my_element = copper then ...`), but most other languages (including Pascal, C, and their successors)

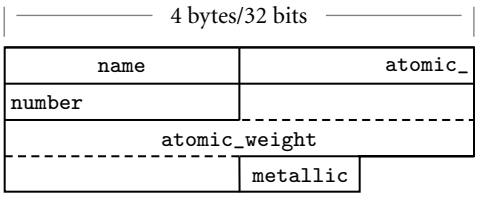


Figure 7.2 Likely memory layout for packed `element` records. The `atomic_number` and `atomic_weight` fields are nonaligned, and can only be read or written (on most machines) via multi-instruction sequences.

do not, though C++ allows the programmer to define equality tests for individual record types. ■

For small records, both copies and comparisons can be performed in-line on a field-by-field basis. For longer records, we can save significantly on code space by deferring to a library routine. A `block_copy` routine can take source address, destination address, and length as arguments, but the analogous `block_compare` routine would fail on records with different (garbage) data in the holes. One solution is to arrange for all holes to contain some predictable value (e.g., zero), but this requires code at every elaboration point. Another is to have the compiler generate a customized field-by-field comparison routine for every record type. Different routines would be called to compare records of different types. Languages like Pascal and C avoid the whole issue by simply outlawing full-record comparisons.

EXAMPLE 7.45
Minimizing holes by sorting fields

In addition to complicating comparisons, holes in records waste space. Packing eliminates holes, but at potentially heavy cost in access time. A compromise, adopted by some compilers, is to sort a record's fields according to the size of their alignment constraints. All byte-aligned fields might come first, followed by any half-word aligned fields, word-aligned fields, and (if the hardware requires) double-word–aligned fields. For our `element` type, the resulting rearrangement is shown in Figure 7.3. ■

In most cases, reordering of fields is purely an implementation issue: the programmer need not be aware of it, so long as all instances of a record type are

DESIGN & IMPLEMENTATION

The order of record fields

Issues of record field order are intimately tied to implementation tradeoffs: Holes in records waste space, but alignment makes for faster access. If holes contain garbage we can't compare records by looping over words or bytes, but zeroing out the holes would incur costs in time and code space. Predictable layout is important for mirroring hardware structures in "systems" languages, but reorganization may be advantageous in large records if we can group frequently accessed fields together, so they lie in the same cache line.

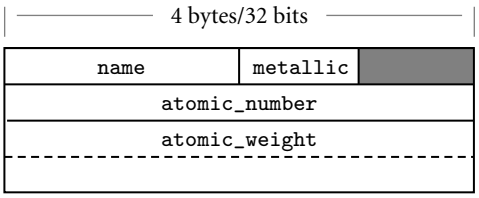


Figure 7.3 Rearranging record fields to minimize holes. By sorting fields according to the size of their alignment constraint, a compiler can minimize the space devoted to holes, while keeping the fields aligned.

reordered in the same way. The exception occurs in systems programs, which sometimes "look inside" the implementation of a data type with the expectation that it will be mapped to memory in a particular way. A kernel programmer, for example, may count on a particular layout strategy in order to define a record that mimics the organization of memory-mapped control registers for a particular Ethernet device. C and C++, which are designed in large part for systems programs, guarantee that the fields of a `struct` will be allocated in the order declared. The first field is guaranteed to have the coarsest alignment required by the hardware for any type (generally a four- or eight-byte boundary). Subsequent fields have the natural alignment for their type. Fortran 90 allows the programmer to specify that fields must not be reordered; in the absence of such a specification the compiler can choose its own order. To accommodate systems programs, Ada and C++ allow the programmer to specify nonstandard alignment for the fields of specific record types.

7.3.3 `With` Statements

EXAMPLE 7.46
Pascal `with` statement

In programs with complicated data structures, manipulating the fields of a deeply nested record can be awkward:

```
ruby.chemical_composition.elements[1].name := 'Al';
ruby.chemical_composition.elements[1].atomic_number := 13;
ruby.chemical_composition.elements[1].atomic_weight := 26.98154;
ruby.chemical_composition.elements[1].metallic := true;
```

Pascal provides a `with` statement to simplify such constructions:

```
with ruby.chemical_composition.elements[1] do begin
    name := 'Al';
    atomic_number := 13;
    atomic_weight := 26.98154;
    metallic := true
end;
```

IN MORE DEPTH

Pascal `with` statements are examined in more detail on the PLP CD. They are generally considered an improvement on the earlier *elliptical references* of Cobol and PL/I. They still suffer from several limitations, however, most of which are addressed in Modula-3 and Fortran 2003. Similar functionality can be achieved with nested scopes in languages like Lisp and ML (which use a reference model of variables), and in languages like C and C++, which allow the programmer to create pointers or references to arbitrary objects.

7.3.4 Variant Records (Unions)

Programming languages of the 1960s and 1970s were designed in an era of severe memory constraints. Many allowed the programmer to specify that certain variables (presumably ones that would never be used at the same time) should be allocated "on top of" one another, sharing the same bytes in memory. C's syntax, heavily influenced by Algol 68, looks very much like a struct:

```
union {
    int i;
    double d;
    _Bool b;
};
```

The overall size of this *union* would be that of its largest member (presumably `d`). Exactly which bytes of `d` would be overlapped by `i` and `b` is implementation dependent, and presumably influenced by the relative sizes of types, their alignment constraints, and the endian-ness of the hardware.

In practice, unions have been used for two main purposes. The first arises in systems programs, where unions allow the same set of bytes to be interpreted in different ways at different times. The canonical example occurs in memory management, where storage may sometimes be treated as unallocated space (perhaps in need of "zeroing out"), sometimes as bookkeeping information (length and header fields to keep track of free and allocated blocks), and sometimes as user-allocated data of arbitrary type. While nonconverting type casts can be used to implement heap management routines, as described on page 309, unions are a better indication of the programmer's intent: the bits are not being reinterpreted, they are being used for independent purposes.[6]

EXAMPLE 7.47

Motivation for variant records

The second common purpose for unions is to represent alternative sets of fields within a record. A record representing an employee, for example, might

6 By contrast, the other example on page 309—examination of the internal structure of a floating-point number—does indeed reinterpret bits. Unions can also be used in this case (Exercise 7.30), but here a nonconverting cast is a better indication of intent.

have several common fields (name, address, phone, department, ID number) and various other fields depending on whether the person in question works on a salaried, hourly, or consulting basis. C unions are awkward when used for this purpose. A much cleaner syntax appears in the *variant records* of Pascal and its successors, which allow the programmer to specify that certain (potentially hierarchical) sets of fields should overlap one another in memory. ■

IN MORE DEPTH

We discuss unions and variant records in more detail on the PLP CD. Topics we consider include syntax, safety, and memory layout issues. Safety is a particular concern: where nonconverting type casts allow a programmer to circumvent the language's type system explicitly, a naive realization of unions makes it easy to do so by accident. Algol 68 and Ada impose limits on the use of unions and variant records that allow the compiler to verify, statically, that all programs are type-safe. We also note that inheritance in object-oriented languages provides an attractive alternative to type-safe variant records in most cases. This observation largely accounts for the omission of unions and variant records from more recent languages.

CHECK YOUR UNDERSTANDING

20. What are *struct tags* in C? How are they related to type names? How did they change in C++?
21. Summarize the distinction between records and *tuples* in ML. How do these compare to the records of languages like C and Ada?
22. Discuss the significance of "holes" in records. Why do they arise? What problems do they cause?
23. Why is it easier to implement assignment than comparison for records?
24. What is *packing*? What are its advantages and disadvantages?
25. Why might a compiler reorder the fields of a record? What problems might this cause?
26. Briefly describe two purposes for unions/variant records.

7.4 Arrays

Arrays are the most common and important composite data types. They have been a fundamental part of almost every high-level language, beginning with Fortran I. Unlike records, which group related fields of disparate types, arrays are usually

homogeneous. Semantically, they can be thought of as a mapping from an *index type* to a *component* or *element type*. Some languages (e.g., Fortran) require that the index type be `integer`; many languages allow it to be any discrete type. Some languages (e.g., Fortran 77) require that the element type of an array be scalar. Most (including Fortran 90) allow any element type.

Some languages (notably scripting languages) allow nondiscrete index types. The resulting *associative* arrays must generally be implemented with hash tables or search trees; we consider them in Section 13.4.3. Associative arrays also resemble the *dictionary* or *map* types supported by the standard libraries of many object-oriented languages. In C++, operator overloading allows these types to use conventional array-like syntax. For the purposes of this chapter, we will assume that array indices are discrete. This admits a (much more efficient) contiguous allocation scheme, to be described in Section 7.4.3.

7.4.1 Syntax and Operations

Most languages refer to an element of an array by appending a subscript—delimited by parentheses or square brackets—to the name of the array. In Fortran and Ada, one says `A(3)`; in Pascal and C, one says `A[3]`. Since parentheses are generally used to delimit the arguments to a subroutine call, square bracket subscript notation has the advantage of distinguishing between the two. The difference in notation makes a program easier to compile and, arguably, easier to read. Fortran's use of parentheses for arrays stems from the absence of square bracket characters on IBM keypunch machines, which at one time were widely used to enter Fortran programs. Ada's use of parentheses represents a deliberate decision on the part of the language designers to embrace notational ambiguity for functions and arrays. If we think of an array as a mapping from the index type to the element type, it makes perfectly good sense to use the same notation used for functions. In some cases, a programmer may even choose to change from an array to a function-based implementation of a mapping, or vice versa (Exercise 7.10).

Declarations

EXAMPLE 7.48
Array declarations

In some languages one declares an array by appending subscript notation to the syntax that would be used to declare a scalar. In C:

```
char upper[26];
```

In Fortran:

```
character, dimension (1:26) :: upper
character (26) upper    ! shorthand notation
```

In C, the lower bound of an index range is always zero: the indices of an n-element array are $0 \ldots n-1$. In Fortran, the lower bound of the index range is one by

default. Fortran 90 allows a different lower bound to be specified if desired, using the notation shown in the first of the two declarations above.

In other languages, arrays are declared with an `array` constructor. In Pascal:

```
var upper : array ['a'..'z'] of char;
```

In Ada:

```
upper : array (character range 'a'..'z') of character;
```

EXAMPLE 7.49
Multidimensional arrays

Most languages make it easy to declare multidimensional arrays:

```
mat : array (1..10, 1..10) of real;     -- Ada

real, dimension (10,10) :: mat          ! Fortran
```

DESIGN & IMPLEMENTATION

Is `[]` an operator?

Associative arrays in C++ are typically defined by overloading `operator[]`. C#, like C++, provides extensive facilities for operator overloading, but it does not use these facilities to support associative arrays. Instead, the language provides a special *indexer* mechanism, with its own unique syntax:

```
class directory {
    Hashtable table;                        // from standard library
    ...
    public directory() {                    // constructor
        table = new Hashtable();
    }
    ...
    public string this[string name] {       // indexer method
        get {
            return (string) table[name];
        }
        set {
            table[name] = value;            // value is implicitly
}   }   }                                   // a parameter of set
...
directory d = new directory();
...
d["Jane Doe"] = "234-5678";
Console.WriteLine(d["Jane Doe"]);
```

Why the difference? In C++, `operator[]` can return a reference (an explicit l-value—see Section 8.3.1), which can be used on either side of an assignment. C# has no comparable notion of reference, so it needs separate methods to `get` and `set` the value of `d["Jane Doe"]`.

In some languages (e.g., Pascal, Ada, and Modula-3), one can also declare a multidimensional array by using the `array` constructor more than once in the same declaration. In Modula-3,

```
VAR mat : ARRAY [1..10], [1..10] OF REAL;
```

is syntactic sugar for

```
VAR mat : ARRAY [1..10] OF ARRAY [1..10] OF REAL;
```

and `mat[3, 4]` is syntactic sugar for `mat[3][4]`. Similar equivalences hold in Pascal.

EXAMPLE 7.50
Multidimensional vs built-up arrays

In Ada, by contrast,

```
mat1 : array (1..10, 1..10) of real;
```

is not the same as

```
type vector is array (integer range <>) of real;
type matrix is array (integer range <>) of vector (1..10);
mat2 : matrix (1..10);
```

Variable `mat1` is a two-dimensional array; `mat2` is an array of one-dimensional arrays. With the former declaration, we can access individual real numbers as `mat1(3, 4)`; with the latter we must say `mat2(3)(4)`. The two-dimensional array is arguably more elegant, but the array of arrays supports additional operations: it allows us to name the rows of `mat2` individually (`mat2(3)` is a 10-element, single-dimensional array), and it allows us to take *slices*, as discussed below (`mat2(3)(2..6)` is a five-element array of real numbers; `mat2(3..7)` is a five-element array of ten-element arrays).

EXAMPLE 7.51
Arrays of arrays in C

In C, one must also declare an array of arrays, and use two-subscript notation, but C's integration of pointers and arrays (to be discussed in Section 7.7.1) means that slices are not supported.

```
double mat[10][10];
```

Given this definition, `mat[3][4]` denotes an individual element of the array, but `mat[3]` denotes a *reference*, either to the third row of the array or to the first element of that row, depending on context.

Slices and Array Operations

EXAMPLE 7.52
Array slice operations

A *slice* or *section* is a rectangular portion of an array. Fortran 90 and Single Assignment C provide extensive facilities for slicing, as do many scripting languages, including Perl, Python, Ruby, and R. Figure 7.4 illustrates some of the possibilities in Fortran 90, using the declaration of `mat` from Example 7.49.

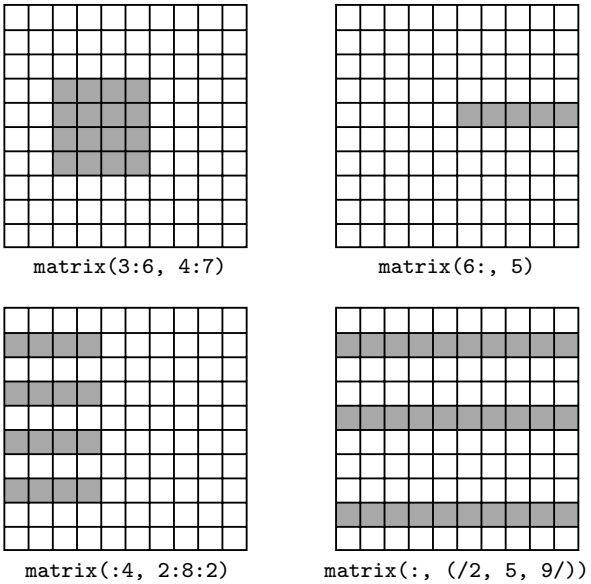


Figure 7.4 **Array slices (sections) in Fortran 90.** Much like the values in the header of an enumeration-controlled loop (Section 6.5.1), $a : b : c$ in a subscript indicates positions a, $a + c$, $a + 2c, \ldots$ through b. If a or b is omitted, the corresponding bound of the array is assumed. If c is omitted, `1` is assumed. It is even possible to use negative values of c in order to select positions in reverse order. The slashes in the second subscript of the lower right example delimit an explicit list of positions.

Ada provides more limited support: a slice is simply a contiguous range of elements in a one-dimensional array. As we saw in Example 7.50, the elements can themselves be arrays, but there is no way to extract a slice along both dimensions as a single operation.

In most languages, the only operations permitted on an array are selection of an element (which can then be used for whatever operations are valid on its type), and assignment. A few languages (e.g., Ada and Fortran 90) allow arrays to be compared for equality. Ada allows one-dimensional arrays whose elements are discrete to be compared for *lexicographic ordering*: `A < B` if the first element of `A` that is not equal to the corresponding element of `B` is less than that corresponding element. Ada also allows the built-in logical operators (`or`, `and`, `xor`) to be applied to Boolean arrays.

Fortran 90 has a very rich set of *array operations*: built-in operations that take entire arrays as arguments. Because Fortran uses structural type equivalence, the operands of an array operator need only have the same element type and shape. In particular, slices of the same shape can be intermixed in array operations, even if the arrays from which they were sliced have very different shapes. Any of the built-in arithmetic operators will take arrays as operands; the result is an array,

of the same shape as the operands, whose elements are the result of applying the operator to corresponding elements. As a simple example, `A + B` is an array each of whose elements is the sum of the corresponding elements of `A` and `B`. Fortran 90 also provides a huge collection of *intrinsic*, or built-in functions. More than 60 of these (including logic and bit manipulation, trigonometry, logs and exponents, type conversion, and string manipulation) are defined on scalars, but will also perform their operation element-wise if passed arrays as arguments. The function `tan(A)`, for example, returns an array consisting of the tangents of the elements of `A`. Many additional intrinsic functions are defined solely on arrays. These include searching and summarization, transposition, and reshaping and subscript permutation.

An equally rich set of array operations can be found in Single Assignment C (SAC), a purely functional language for high-performance computing developed by Sven-Bodo Scholz and others in the mid to late 1990s, and currently in active use at a variety of sites. Both SAC and Fortran 90 take significant inspiration from APL, an array manipulation language developed by Iverson and others in the early to mid-1960s.[7] APL was designed primarily as a terse mathematical notation for array manipulations. It employs an enormous character set that made it difficult to use with traditional keyboards and textual displays. Its variables are all arrays, and many of the special characters denote array operations. APL implementations are designed for interpreted, interactive use. They are best suited to "quick and dirty" solution of mathematical problems. The combination of very powerful operators with very terse notation makes APL programs notoriously difficult to read and understand. The J notation, a successor to APL, uses a conventional character set.

7.4.2 Dimensions, Bounds, and Allocation

In all of the examples in the previous subsection, the shape of the array (including bounds) was specified in the declaration. For such static shape arrays, storage can be managed in the usual way: static allocation for arrays whose lifetime is the entire program; stack allocation for arrays whose lifetime is an invocation of a subroutine; heap allocation for dynamically allocated arrays with more general lifetime.

Storage management is more complex for arrays whose shape is not known until elaboration time, or whose shape may change during execution. For these the compiler must arrange not only to allocate space, but also to make shape information available at run time (without such information, indexing would not be possible). Some dynamically typed languages allow run-time binding of

---

7 Kenneth Iverson (1920–2004), a Canadian mathematician, joined the faculty at Harvard University in 1954, where he conceived APL as a notation for describing mathematical algorithms. He moved to IBM in 1960, where he helped develop the notation into a practical programming language. He was named an IBM Fellow in 1970, and received the ACM Turing Award in 1979.

both the number and bounds of dimensions. Compiled languages may allow the bounds to be dynamic, but typically require the number of dimensions to be static. A local array whose shape is known at elaboration time may still be allocated in the stack. An array whose size may change during execution must generally be allocated in the heap.

In the first subsection below we consider the *descriptors*, or *dope vectors*,[8] used to hold shape information at run time. We then consider stack- and heap-based allocation, respectively, for dynamic shape arrays.

Dope Vectors

During compilation, the symbol table maintains dimension and bounds information for every array in the program. For every record, it maintains the offset of every field. When the number and bounds of array dimensions are statically known, the compiler can look them up in the symbol table in order to compute the address of elements of the array. When these values are not statically known, the compiler must generate code to look them up in a dope vector at run time.

Typically, a dope vector will contain the lower bound of each dimension and the size of each dimension other than the last (which will always be the size of the element type, and will thus be statically known). If the language implementation performs dynamic semantic checks for out-of-bounds subscripts in array references, then the dope vector may contain upper bounds as well. Given lower bounds and sizes, the upper bound information is redundant, but it is usually included anyway, to avoid computing it repeatedly at run time.

The contents of the dope vector are initialized at elaboration time, or whenever the number or bounds of dimensions change. In a language like Fortran 90, whose notion of shape includes dimension sizes but not lower bounds, an assignment statement may need to copy not only the data of an array, but dope vector contents as well.

In a language that provides both a value model of variables and arrays of dynamic shape, we must consider the possibility that a record will contain a field whose size is not statically known. In this case the compiler may use dope vectors not only for dynamic shape arrays, but also for dynamic shape records. The dope vector for a record typically indicates the offset of each field from the beginning of the record.

Stack Allocation

EXAMPLE 7.53
Conformant array parameters in Pascal

Subroutine parameters are the simplest example of dynamic shape arrays. Early versions of Pascal required the shape of all arrays to be specified statically. Standard Pascal relaxes this requirement by allowing array parameters to have bounds that

8 The name "dope vector" presumably derives from the notion of "having the dope on (something)," a colloquial expression that originated in horse racing: advance knowledge that a horse has been drugged ("doped") is of significant, if unethical, use in placing bets.

are symbolic names rather than constants. It calls these parameters *conformant arrays*:

```
function DotProduct(A, B:array [lower..upper:integer] of real):real;
var i : integer;
    rtn : real;
begin
    rtn := 0;
    for i := lower to upper do rtn := rtn + A[i] * B[i];
    DotProduct := rtn
end;
```

Here `lower` and `upper` are initialized at the time of call, providing `DotProduct` with the information it needs to understand the shape of `A` and `B`. In effect, `lower` and `upper` are extra parameters of `DotProduct`.

Conformant arrays are highly useful in scientific applications, many of which rely on numerical libraries for linear algebra and the manipulation of systems of equations. Since different programs use arrays of different shapes, the subroutines in these libraries need to be able to take arguments whose size is not known at compile time.

Pascal allows conformant arrays to be passed by reference or by value (Section 8.3.1). In either case, the caller passes both the data of the array and an appropriate dope vector, in which `lower` and `upper` can be found. If the array is of dynamic shape in the caller's context, the dope vector will already exist. If the array is of static shape in the caller's context, the caller will need to create an appropriate dope vector prior to the call. ■

Conformant arrays appear in several other languages. Modula-2 inherits them from Pascal. C supports single-dimensional arrays of dynamic shape as a natural consequence of its merger of arrays and pointers (to be discussed in Section 7.7.1). Ada and C99 support not only conformant arrays, but also *local* arrays of dynamic shape. Among other things, local arrays can be declared to match the shape of conformant array parameters, facilitating the implementation of algorithms that require temporary space for calculations. Figure 7.5 contains a simple example in C99. Function `square` accepts an array parameter `M` of dynamic shape and allocates a local variable `T` of the same dynamic shape. ■

EXAMPLE 7.54
Local arrays of dynamic shape in C99

In many languages, including Ada and C99, the shape of a local array becomes fixed at elaboration time. For such arrays it is still possible to place the space for the array in the stack frame of its subroutine, but an extra level of indirection is required (see Figure 7.6). In order to ensure that every local object can be found using a known offset from the frame pointer, we divide the stack frame into a *fixed-size part* and a *variable-size part*. An object whose size is statically known goes in the fixed-size part. An object whose size is not known until elaboration time goes in the variable-size part, and a pointer to it, together with a dope vector, goes in the fixed-size part. If the elaboration of the array is buried in a nested block, the compiler delays allocating space (i.e., changing the stack pointer) until the block is entered. It still allocates space for the pointer and the dope vector among the

EXAMPLE 7.55
Stack allocation of elaborated arrays

```
void square(int n, double M[n][n]) {
    double T[n][n];
    for (int i = 0; i < n; i++) {       // compute product into T
        for (int j = 0; j < n; j++) {
            double s = 0;
            for (int k = 0; k < n; k++) {
                s += M[i][k] * M[k][j];
            }
            T[i][j] = s;
        }
    }
    for (int i = 0; i < n; i++) {       // copy back into M
        for (int j = 0; j < n; j++) {
            M[i][j] = T[i][j];
        }
    }
}
```

Figure 7.5 A dynamic local array in C99. Function `square` multiplies a matrix by itself and replaces the original with the product. To do so it needs a scratch array of the same shape as the parameter. Note that the declarations of `M` and `T` both rely on parameter `n`.

local variables when the subroutine itself is entered. Records of dynamic shape are handled in a similar way. ■

EXAMPLE 7.56
Elaborated arrays in Fortran 90

Fortran 90 allows specification of the bounds of an array to be delayed until after elaboration, but it does not allow those bounds to change once they have been defined:

```
real, dimension (:,:), allocatable :: mat
    ! mat is two-dimensional, but with unspecified bounds
...
allocate (mat (a:b, 0:m-1))
    ! first dimension has bounds a..b; second has bounds 0..m-1
...
deallocate (mat)
    ! implementation is now free to reclaim mat's space
```

Execution of an `allocate` statement can be treated like the elaboration of a dynamic shape array in a nested block. Execution of a `deallocate` statement can be treated like the end of the nested block (restoring the previous stack pointer) if there are no other arrays beyond the specified one in the stack. Alternatively, dynamic shape arrays can be allocated in the stack, as described in the following subsection. ■

Heap Allocation

Arrays that can change shape at arbitrary times are sometimes said to be *fully dynamic*. Because changes in size do not in general occur in FIFO order, stack allocation will not suffice; fully dynamic arrays must be allocated in the heap.

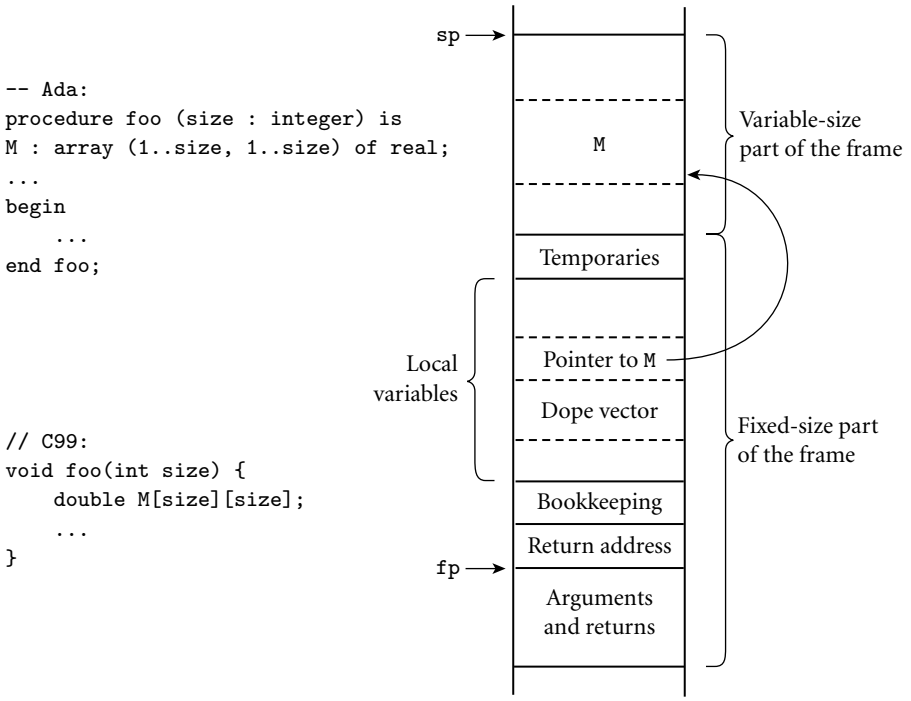


Figure 7.6 **Elaboration-time allocation of arrays in Ada or C99.** Here `M` is a square two-dimensional array whose bounds are determined by a parameter passed to `foo` at run time. The compiler arranges for a pointer to `M` and a dope vector to reside at static offsets from the frame pointer. `M` cannot be placed among the other local variables because it would prevent those higher in the frame from having static offsets. Additional variable-size arrays or records are easily accommodated.

EXAMPLE 7.57
Dynamic strings in Java and C#

Several languages, including Snobol, Icon, and all the scripting languages, allow strings—arrays of characters—to change size after elaboration time. Java and C# provide a similar capability (with a similar implementation), but describe the semantics differently: string variables in these languages are references to immutable string objects:

```
String s = "short";    // This is Java; use lowercase 'string' in C#
...
s = s + " but sweet";  // + is the concatenation operator
```

Here the declaration `String s` introduces a string variable, which we initialize with a reference to the constant string `"short"`. In the subsequent assignment, `+` creates a new string containing the concatenation of the old `s` and the constant `" but sweet"`; `s` is then set to refer to this new string, rather than the old.

Java and C# strings, by the way, are *not* the same as arrays of characters: strings are immutable, but elements of an array can be changed in place. ■

Dynamically resizable arrays (other than strings) appear in APL, Common Lisp, and the various scripting languages. They are also supported by the `vector`, `Vector`, and `ArrayList` classes of the C++, Java, and C# libraries, respectively. In contrast to the `allocate`-able arrays of Fortran 90, these arrays can change their shape—in particular, can grow—while retaining their current content. In most cases, increasing the size will require that the run-time system allocate a larger block, copy any data that are to be retained from the old block to the new, and then deallocate the old.

If the number of dimensions of a fully dynamic array is statically known, the dope vector can be kept, together with a pointer to the data, in the stack frame of the subroutine in which the array was declared. If the number of dimensions can change, the dope vector must generally be placed at the beginning of the heap block instead.

In the absence of garbage collection, the compiler must arrange to reclaim the space occupied by fully dynamic arrays when control returns from the subroutine in which they were declared. Space for stack-allocated arrays is of course reclaimed automatically by popping the stack.

7.4.3 Memory Layout

Arrays in most language implementations are stored in contiguous locations in memory. In a one-dimensional array, the second element of the array is stored immediately after the first (subject to alignment constraints); the third is stored immediately after the second, and so forth. For arrays of records, it is common for each subsequent element to be aligned at an address appropriate for any type; small holes between consecutive records may result.

EXAMPLE 7.58
Row-major vs column-major array layout

For multidimensional arrays, it still makes sense to put the first element of the array in the array's first memory location. But which element comes next? There are two reasonable answers, called *row-major* and *column-major* order. In row-major order, consecutive locations in memory hold elements that differ by one in the final subscript (except at the ends of rows). `A[2, 4]`, for example, is followed by `A[2, 5]`. In column-major order, consecutive locations hold elements that differ by one in the *initial* subscript: `A[2, 4]` is followed by `A[3, 4]`. These options are illustrated for two-dimensional arrays in Figure 7.7. The layouts for three or more dimensions are analogous. Fortran uses column-major order; most other languages use row-major order. (Correspondence with Fran Allen[9]

9 Fran Allen (1932–) joined IBM's T. J. Watson Research Center in 1957, and stayed for her entire professional career. Her seminal paper, *Program Optimization* [All69] helped launch the field of code improvement. Her PTRAN (Parallel TRANslation) group, founded in the early 1980s, developed much of the theory of automatic parallelization. In 1989 Dr. Allen became the first woman to be named in IBM Fellow. In 2006 she became the first to receive the ACM Turing Award.

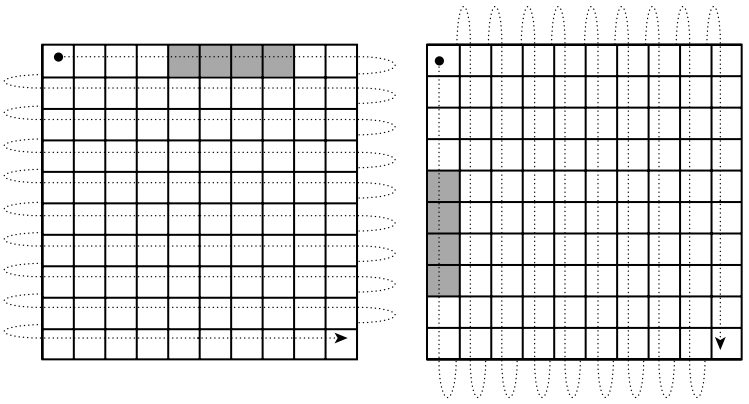


Figure 7.7 **Row- and column-major memory layout for two-dimensional arrays.** In row-major order, the elements of a row are contiguous in memory; in column-major order, the elements of a column are contiguous. The second cache line of each array is shaded, on the assumption that each element is an eight-byte floating-point number, that cache lines are 32 bytes long (a common size), and that the array begins at a cache line boundary. If the array is indexed from A[0,0] to A[9,9], then in the row-major case elements A[0,4] through A[0,7] share a cache line; in the column-major case elements A[4,0] through A[7,0] share a cache line.

suggests that column-major order was originally adopted in order to accommodate idiosyncrasies of the console debugger and instruction set of the IBM model 704 computer, on which the language was first implemented.) The advantage of row-major order is that it makes it easy to define a multidimensional array as an array of subarrays, as described in Section 7.4.1. With column-major order, the elements of the subarray would not be contiguous in memory. ■

EXAMPLE 7.59
Array layout and cache performance

The difference between row- and column-major layout can be important for programs that use nested loops to access all the elements of a large, multidimensional array. On modern machines the speed of such loops is often limited by memory system performance, which depends heavily on the effectiveness of caching (Section ©5.1). Figure 7.7 shows the orientation of cache lines for row- and column-major layout of arrays. When code traverses a small array, all or most of its elements are likely to remain in the cache through the end of the nested loops, and the orientation of cache lines will not matter. For a large array, however, lines that are accessed early in the traversal are likely to be evicted to make room for lines accessed later in the traversal. If array elements are accessed in order of consecutive addresses, then each miss will bring into the cache not only the desired element, but the next several elements as well. If elements are accessed *across* cache lines instead (i.e., along the rows of a Fortran array, or the columns of an array in most other languages), then there is a good chance that almost every access will result in a cache miss, dramatically reducing the performance of the code. In C, one should write

```
for (i = 0; i < N; i++) {        /* rows */
    for (j = 0; j < N; j++) {    /* columns */
        ... A[i][j] ...
    }
}
```

In Fortran:

```
do j = 1, N                      ! columns
    do i = 1, N                  ! rows
        ... A(i, j) ...
    end do
end do
```

Row-Pointer Layout

Some languages employ an alternative to contiguous allocation for some arrays. Rather than require the rows of an array to be adjacent, they allow them to lie anywhere in memory, and create an auxiliary array of pointers to the rows. If the array has more than two dimensions, it may be allocated as an array of pointers to arrays of pointers to. . . . This *row-pointer* memory layout requires more space in most cases, but has three potential advantages. First, it sometimes allows individual elements of the array to be accessed more quickly, especially on CISC machines with slow multiplication instructions (see the discussion of address calculations below). Second, it allows the rows to have different lengths, without devoting space to holes at the ends of the rows. This representation is sometimes called a *ragged array*. The lack of holes may sometimes offset the increased space for pointers. Third, it allows a program to construct an array from preexisting rows (possibly scattered throughout memory) without copying. C, C++, and C# provide both contiguous and row-pointer organizations for multidimensional arrays. Technically speaking, the contiguous layout is a true multidimensional array, while the row-pointer layout is an array of pointers to arrays. Java uses the row-pointer layout for all arrays.

DESIGN & IMPLEMENTATION

Array layout

The layout of arrays in memory, like the ordering of record fields, is intimately tied to tradeoffs in design and implementation. While column-major layout appears to offer no advantages on modern machines, its continued use in Fortran means that programmers must be aware of the underlying implementation in order to achieve good locality in nested loops. Row-pointer layout, likewise, has no performance advantage on modern machines (and a likely performance *penalty*, at least for numeric code), but it is a more natural fit for the "reference to object" data organization of languages like Java. Its impacts on space consumption and locality may be positive or negative, depending on the details of individual applications.

```
char days[][10] = {
    "Sunday", "Monday", "Tuesday",
    "Wednesday", "Thursday",
    "Friday", "Saturday"
};
...
days[2][3] == 's';  /* in Tuesday */
```

```
char *days[] = {
    "Sunday", "Monday", "Tuesday",
    "Wednesday", "Thursday",
    "Friday", "Saturday"
};
...
days[2][3] == 's';  /* in Tuesday */
```

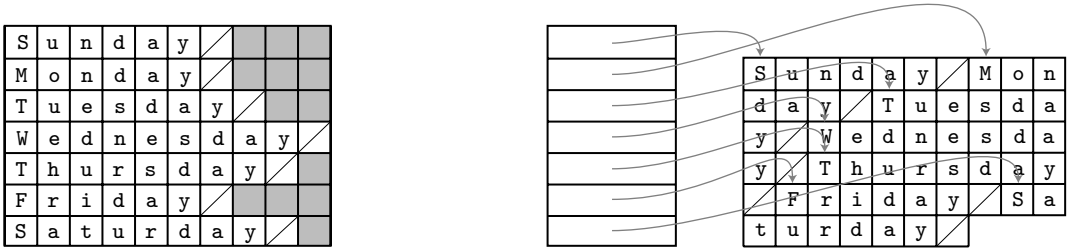


Figure 7.8 **Contiguous array allocation vs row pointers in C.** The declaration on the left is a true two-dimensional array. The slashed boxes are `NUL` bytes; the shaded areas are holes. The declaration on the right is a ragged array of pointers to arrays of characters. In both cases, we have omitted bounds in the declaration that can be deduced from the size of the initializer (aggregate). Both data structures permit individual characters to be accessed using double subscripts, but the memory layout (and corresponding address arithmetic) is quite different.

EXAMPLE 7.60
Contiguous vs row-pointer array layout

By far the most common use of the row-pointer layout in C is to represent arrays of strings. A typical example appears in Figure 7.8. In this example (representing the days of the week), the row-pointer memory layout consumes 57 bytes for the characters themselves (including a `NUL` byte at the end of each string), plus 28 bytes for pointers (assuming a 32-bit architecture), for a total of 85 bytes. The contiguous layout alternative devotes 10 bytes to each day (room enough for `Wednesday` and its `NUL` byte), for a total of 70 bytes. The additional space required for the row-pointer organization comes to 21%. In other cases, row pointers may actually save space. A Java compiler written in C, for example, would probably use row pointers to store the character-string representations of the 51 Java keywords and word-like literals. This data structure would use $51 \times 4 = 204$ bytes for the pointers, plus 343 bytes for the keywords, for a total of 547 bytes (548 when aligned). Since the longest keyword (`synchronized`) requires 13 bytes (including space for the terminating `NUL`), a contiguous two-dimensional array would consume $51 \times 13 = 663$ bytes (664 when aligned). In this case, row pointers *save* a little over 21%. ■

Address Calculations

EXAMPLE 7.61
Indexing a contiguous array

For the usual contiguous layout of arrays, calculating the address of a particular element is somewhat complicated, but straightforward. Suppose a compiler is given the following declaration for a three-dimensional array:

A : array $[L_1 \,..\, U_1]$ of array $[L_2 \,..\, U_2]$ of array $[L_3 \,..\, U_3]$ of elem_type;

Let us define constants for the sizes of the three dimensions:

$$\begin{aligned} S_3 &= \text{size of elem\_type} \\ S_2 &= (U_3 - L_3 + 1) \times S_3 \\ S_1 &= (U_2 - L_2 + 1) \times S_2 \end{aligned}$$

Here the size of a row (S_2) is the size of an individual element (S_3) times the number of elements in a row (assuming row-major layout). The size of a plane (S_1) is the size of a row (S_2) times the number of rows in a plane. The address of A[i, j, k] is then

$$\begin{aligned} &\text{address of A} \\ &+ \ (i - L_1) \times S_1 \\ &+ \ (j - L_2) \times S_2 \\ &+ \ (k - L_3) \times S_3 \end{aligned}$$

As written, this computation involves three multiplications and six additions/subtractions. We could compute the entire expression at run time, but in most cases a little rearrangement reveals that much of the computation can be performed at compile time. In particular, if the bounds of the array are known at compile time, then S_1, S_2, and S_3 are compile-time constants, and the subtractions of lower bounds can be distributed out of the parentheses:

$$\begin{aligned} &(i \times S_1) + (j \times S_2) + (k \times S_3) + \text{address of A} \\ &\quad -[(L_1 \times S_1) + (L_2 \times S_2) + (L_3 \times S_3)] \end{aligned}$$

The bracketed expression in this formula is a compile-time constant (assuming the bounds of A are statically known). If A is a global variable, then the address of A is statically known as well, and can be incorporated in the bracketed expression. If A is a local variable of a subroutine (with static shape), then the address of A can be decomposed into a static offset (included in the bracketed expression) plus the contents of the frame pointer at run time. We can think of the address of A plus the bracketed expression as calculating the location of an imaginary array whose [i, j, k]th element coincides with that of A, but whose lower bound in each dimension is zero. This imaginary array is illustrated in Figure 7.9. ■

EXAMPLE 7.62

Pseudo-assembler for contiguous array indexing

If A's elements are integers, and are allocated contiguously in memory, then the instruction sequence to load A[i, j, k] into a register looks something like this:

```
    -- assume i is in r1, j is in r2, and k is in r3
1.  r4 := r1 × S1
2.  r5 := r2 × S2
3.  r6 := &A – L1 × S1 – L2 × S2 – L3 × 4      -- one or two instructions
4.  r6 := r6 + r4
5.  r6 := r6 + r5
6.  r7 := *r6[r3]                              -- load
```

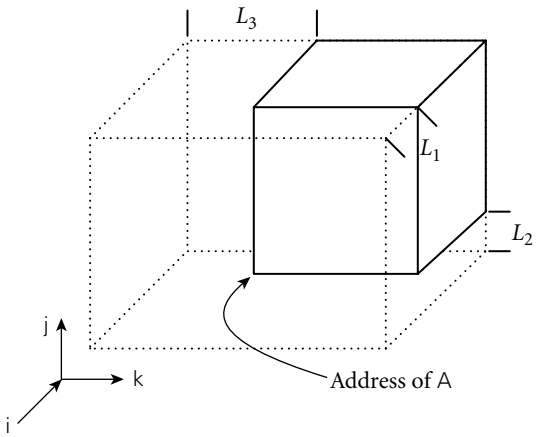


Figure 7.9 **Virtual location of an array with nonzero lower bounds.** By computing the constant portions of an array index at compile time, we effectively index into an array whose starting address is offset in memory, but whose lower bounds are all zero.

We have assumed that the hardware provides an indexed addressing mode, and that it scales its indexing by the size of the quantity loaded (in this case a four-byte integer). ■

EXAMPLE 7.63
Static and dynamic portions of an array index

If i, j, and/or k is known at compile time, then additional portions of the calculation of the address of A[i, j, k] will move from the dynamic to the static part of the formula shown above. If all of the subscripts are known, then the entire address can be calculated statically. Conversely, if any of the bounds of the array are not known at compile time, then portions of the calculation will move from the static to the dynamic part of the formula. For example, if L_1 is not known until run time, but k is known to be 3 at compile time, then the calculation becomes

$$(i \times S_1) + (j \times S_2) - (L_1 \times S_1) + \text{address of A} - [(L_2 \times S_2) + (L_3 \times S_3) - (3 \times S_3)]$$

Again, the bracketed part can be computed at compile time. If lower bounds are always restricted to zero, as they are in C, then they never contribute to run-time cost. ■

In all our examples, we have ignored the issue of dynamic semantic checks for out-of-bound subscripts. We explore the code for these in Exercise 7.18. In Section ©16.5.2 we will consider code improvement techniques that can be used to eliminate many checks statically, particularly in enumeration-controlled loops.

EXAMPLE 7.64
Indexing complex structures

The notion of "static part" and "dynamic part" of an address computation generalizes to more than just arrays. Suppose, for example, that V is a messy local array of records containing a nested, two-dimensional array in field M. The address of V[i].M[3, j] could be calculated as

$$
\begin{array}{ll}
i \times S_1^{\mathsf{V}} & \\
 & -L_1^{\mathsf{V}} \times S_1^{\mathsf{V}} \\
 & +\text{M's offset as a field} \\
 & +(3 - L_1^{\mathsf{V}}) \times S_1^{\mathsf{V}} \\
+j \times S_2^{\mathsf{V}} & \\
 & -L_2^{\mathsf{V}} \times S_2^{\mathsf{V}} \\
+\text{fp} & \\
 & +\text{ offset of V in frame}
\end{array}
$$

Here the calculations on the left must be performed at run time; the calculations on the right can be performed at compile time. (The notation for bounds and size places the name of the variable in a superscript and the dimension in a subscript: L_2^{M} is the lower bound of the second dimension of M.)

EXAMPLE 7.65

Pseudo-assembler for row-pointer array indexing

Address calculation for arrays that use row pointers is comparatively straightforward. Using our three-dimensional array A as an example, the expression A[i, j, k] is equivalent, in C notation, to (*(*A[i])[j])[k]. The instruction sequence to load A[i, j, k] into a register looks something like this:

```
-- assume i is in r1, j is in r2, and k is in r3
1. r4 := &A              -- one or two instructions
2. r4 := *r4[r1]
3. r4 := *r4[r2]
4. r7 := r4[r3]
```

Assuming that the loads at lines 2 and 3 hit in the cache, this code will be comparable in cost to the instruction sequence for contiguous allocation shown above

DESIGN & IMPLEMENTATION

Lower bounds on array indices

In C, the lower bound of every array dimension is always zero. It is often assumed that the language designers adopted this convention in order to avoid subtracting lower bounds from indices at run time, thereby avoiding a potential source of inefficiency. As our discussion has shown, however, the compiler can avoid any run-time cost by translating to a virtual starting location. (The one exception to this statement occurs when the lower bound has a very large absolute value: if any index (scaled by element size) exceeds the maximum offset available with displacement mode addressing [typically 2^{15} bytes on RISC machines], then subtraction may still be required at run time.)

A more likely explanation lies in the interoperability of arrays and pointers in C (Section 7.7.1): C's conventions allow the compiler to generate code for an index operation on a pointer without worrying about the lower bound of the array into which the pointer points. Interestingly, Fortran array dimensions have a default lower bound of 1; unless the programmer explicitly specifies a lower bound of 0, the compiler must always translate to a virtual starting location.

(given load delays). If the intermediate loads miss in the cache, it will be slower. On a 1970s CISC machine, the balance would probably tip in favor of the row-pointer code: multiplies would be slower, and memory accesses faster. In any event (contiguous or row-pointer allocation, old or new machine), important code improvements will often be possible when several array references use the same subscript expression, or when array references are embedded in loops. ■

CHECK YOUR UNDERSTANDING

27. What is an array *slice*? For what purposes are slices useful?
28. Is there any significant difference between a two-dimensional array and an array of one-dimensional arrays?
29. What is the *shape* of an array?
30. What is a *dope vector*? What purpose does it serve?
31. Under what circumstances can an array declared within a subroutine be allocated in the stack? Under what circumstances must it be allocated in the heap?
32. What is a *conformant* array?
33. Discuss the comparative advantages of *contiguous* and *row-pointer* layout for arrays.
34. Explain the difference between *row-major* and *column-major* layout for contiguously allocated arrays. Why does a programmer need to know which layout the compiler uses? Why do most language designers consider row-major layout to be better?
35. How much of the work of computing the address of an element of an array can be performed at compile time? How much must be performed at run time?

7.5 Strings

In many languages, a string is simply an array of characters. In other languages, strings have special status, with operations that are not available for arrays of other sorts. Particularly powerful string facilities are found in Snobol, Icon, and the various scripting languages.

As we saw in Section 6.5.4, mechanisms to search for patterns within strings are a key part of Icon's distinctive generator-based control flow. Icon has dozens of built-in string operators, functions, and generators, including sophisticated pattern-matching facilities based on regular expressions. Perl, Python, Ruby, and other scripting languages provide similar functionality, though none includes the full power of Icon's backtracking search. We will consider the string and pattern-matching facilities of scripting languages in more detail in Section 13.4.2. In the

remainder of this section we focus on the role of strings in more traditional languages.

Almost all programming languages allow literal strings to be specified as a sequence of characters, usually enclosed in single or double quote marks. Many languages, including C and its descendants, distinguish between literal characters (usually delimited with single quotes) and literal strings (usually delimited with double quotes). Other languages (e.g., Pascal) make no distinction: a character is just a string of length one. Most languages also provide *escape sequences* that allow nonprinting characters and quote marks to appear inside of strings.

EXAMPLE 7.66
Character escapes in C and C++

C99 and C++ provide a very rich set of escape sequences. An arbitrary character can be represented by a backslash followed by (a) 1 to 3 octal (base-8) digits, (b) an `x` and one or more hexadecimal (base-16) digits, (c) a `u` and exactly four hexadecimal digits, or (d) a `U` and exactly eight hexadecimal digits. The `\U` notation is meant to capture the four-byte (32-bit) Unicode character set described in the sidebar on page 295. The `\u` notation is for characters in the Basic Multilingual Plane. Many of the most common control characters also have single-character escape sequences, many of which have been adopted by other languages as well. For example, `\n` is a line feed; `\t` is a tab; `\r` is a carriage return; `\\` is a backslash. C# omits the octal sequences of C99 and C++; Java also omits the 32-bit extended sequences.

The set of operations provided for strings is strongly tied to the implementation envisioned by the language designer(s). Several languages that do not in general allow arrays to change size dynamically do provide this flexibility for strings. The rationale is twofold. First, manipulation of variable-length strings is fundamental to a huge number of computer applications, and in some sense "deserves" special treatment. Second, the fact that strings are one-dimensional, have one-byte elements, and never contain references to anything else makes dynamic-size strings easier to implement than general dynamic arrays.

Some languages require that the length of a string-valued variable be bound no later than elaboration time, allowing the variable to be implemented as a contiguous array of characters in the current stack frame. Pascal and Ada support a few string operations, including assignment and comparison for lexicographic ordering. C, on the other hand, provides only the ability to create a pointer to a string literal. Because of C's unification of arrays and pointers, even assignment is not supported. Given the declaration `char *s`, the statement `s = "abc"` makes `s` point to the constant `"abc"` in static storage. If `s` is declared as an array, rather than a pointer (`char s[4]`), then the statement will trigger an error message from the compiler. To assign one array into another in C, the program must copy the elements individually.

EXAMPLE 7.67
`Char*` assignment in C

Other languages allow the length of a string-valued variable to change over its lifetime, requiring that the variable be implemented as a block or chain of blocks in the heap. ML and Lisp provide strings as a built-in type. C++, Java, and C# provide them as predefined classes of object, in the formal, object-oriented sense. In all these languages a string variable is a *reference* to a string. Assigning a new value to such a variable makes it refer to a different object. Concatenation and

other string operators implicitly create new objects. The space used by objects that are no longer reachable from any variable is reclaimed automatically.

7.6 Sets

A programming language set is an unordered collection of an arbitrary number of distinct values of a common type. Sets were introduced by Pascal, and are found in many more recent languages as well. The type from which elements of a set are drawn is known as the *base* or *universe* type. Pascal supports sets of any discrete type, and provides union, intersection, and difference operations:

EXAMPLE 7.68
Set types in Pascal

```
var A, B, C : set of char;
    D, E : set of weekday;
...
A := B + C;  (* union; A := {x | x is in B or x is in C} *)
A := B * C;  (* intersection; A := {x | x is in B and x is in C} *)
A := B - C;  (* difference; A := {x | x is in B and x is not in C} *)
```

Icon supports sets of characters (called *csets*), but not sets of any other base type. Python supports sets of arbitrary type; we describe these in Example 13.69 (page 708). Ada does not provide a type constructor for sets, but its generic facility can be used to define a set package (module) with functionality comparable to the sets of Pascal [IBFW91, pp. 242–244]. In a similar vein, sets appear in the standard libraries of many object-oriented languages, including C++, Java, and C#. ■

There are many ways to implement sets, including arrays, hash tables, and various forms of trees. For discrete base types with a modest number of elements,

DESIGN & IMPLEMENTATION

Representing sets

Unfortunately, bit vectors do not work well for large base types: a set of integers, represented as a bit vector, would consume some 500 megabytes on a 32-bit machine. With 64-bit integers, a bit-vector set would consume more memory than is currently contained on all the computers in the world. Because of this problem, many languages (including early versions of Pascal, but not the ISO standard) limit sets to base types of fewer than some fixed number of members. Both 128 and 256 are common limits; they suffice to cover ASCII characters. A few languages (e.g., early versions of Modula-2) limit base types to the number of elements that can be represented by a one-word bit vector, but there is really no excuse for such a severe restriction. A language that permits sets with very large base types must employ an alternative implementation (e.g., a hash table). It will still be expensive to represent sets with enormous numbers of elements, but reasonably easy to represent sets with a modest number of elements drawn from a very large universe.

a *characteristic array* is a particularly appealing implementation: it employs a bit vector whose length (in bits) is the number of distinct values of the base type. A one in the *k*th position in the bit vector indicates that the *k*th element of the base type is a member of the set; a zero indicates that it is not. In a language that uses ASCII, a set of characters would occupy 128 bits—16 bytes. Operations on bit-vector sets can make use of fast logical instructions on most machines. Union is bit-wise `or`; intersection is bit-wise `and`; difference is bit-wise `not`, followed by bit-wise `and`.

7.7 Pointers and Recursive Types

A recursive type is one whose objects may contain one or more references to other objects of the type. Most recursive types are records, since they need to contain something in addition to the reference, implying the existence of heterogeneous fields. Recursive types are used to build a wide variety of "linked" data structures, including lists and trees.

In languages that use a reference model of variables, it is easy for a record of type `foo` to include a reference to another record of type `foo`: every variable (and hence every record field) *is* a reference anyway. In languages that use a value model of variables, recursive types require the notion of a *pointer*: a variable (or field) whose value is a reference to some object. Pointers were first introduced in PL/I.

In some languages (e.g., Pascal, Ada 83, and Modula-3), pointers are restricted to point only to objects in the heap. The only way to create a new pointer value (without using variant records or casts to bypass the type system) is to call a built-in function that allocates a new object in the heap and returns a pointer to it. In other languages (e.g., PL/I, Algol 68, C, C++, and Ada 95), one can create a pointer to a nonheap object by using an "address of" operator. We will examine pointer operations and the ramifications of the reference and value models in more detail in the first subsection below.

In any language that permits new objects to be allocated from the heap, the question arises: how and when is storage reclaimed for objects that are no longer

DESIGN & IMPLEMENTATION

Implementation of pointers

It is common for programmers (and even textbook writers) to equate pointers with addresses, but this is a mistake. A pointer is a high-level concept: a reference to an object. An address is a low-level concept: the location of a word in memory. Pointers are often implemented as addresses, but not always. On a machine with a *segmented* memory architecture, a pointer may consist of a segment id and an offset within the segment. In a language that attempts to catch uses of dangling references, a pointer may contain both an address and an access key.

needed? In short-lived programs it may be acceptable simply to leave the storage unused, but in most cases unused space must be reclaimed, to make room for other things. A program that fails to reclaim the space for objects that are no longer needed is said to "leak memory." If such a program runs for an extended period of time, it may run out of space and crash.

Many languages, including C, C++, Pascal, and Modula-2, require the programmer to reclaim space explicitly. Other languages, including Modula-3, Java, C#, and all the functional and scripting languages, require the language implementation to reclaim unused objects automatically. Explicit storage reclamation simplifies the language implementation, but raises the possibility that the programmer will forget to reclaim objects that are no longer live (thereby leaking memory), or will accidentally reclaim objects that are still in use (thereby creating *dangling references*). Automatic storage reclamation (otherwise known as *garbage collection*) dramatically simplifies the programmer's task, but imposes certain run-time costs, and raises the question of how the language implementation is to distinguish garbage from active objects. We will discuss dangling references and garbage collection further in Sections 7.7.2 and 7.7.3, respectively.

7.7.1 Syntax and Operations

Operations on pointers include allocation and deallocation of objects in the heap, dereferencing of pointers to access the objects to which they point, and assignment of one pointer into another. The behavior of these operations depends heavily on whether the language is functional or imperative, and on whether it employs a reference or value model for variables/names.

Functional languages generally employ a reference model for names (a purely functional language has no variables or assignments). Objects in a functional language tend to be allocated automatically as needed, with a structure determined by the language implementation. Variables in an imperative language may use either a value or a reference model, or some combination of the two. In C, Pascal, or Ada, which employ a value model, the assignment `A := B` puts the value of `B` into `A`. If we want `B` to refer to an object, and we want `A := B` to make `A` refer to the object to which `B` refers, then `A` and `B` must be pointers. In Clu and Smalltalk, which employ a reference model, the assignment `A := B` always makes `A` refer to the same object to which `B` refers.

Java charts an intermediate course, in which the usual implementation of the reference model is made explicit in the language semantics. Variables of built-in Java types (integers, floating-point numbers, characters, and Booleans) employ a value model; variables of user-defined types (strings, arrays, and other objects in the object-oriented sense of the word) employ a reference model. The assignment `A := B` in Java places the value of `B` into `A` if `A` and `B` are of built-in type; it makes `A` refer to the object to which `B` refers if `A` and `B` are of user-defined type. C# mirrors Java by default, but additional language features, explicitly labeled "`unsafe`," allow systems programmers to use pointers when desired.

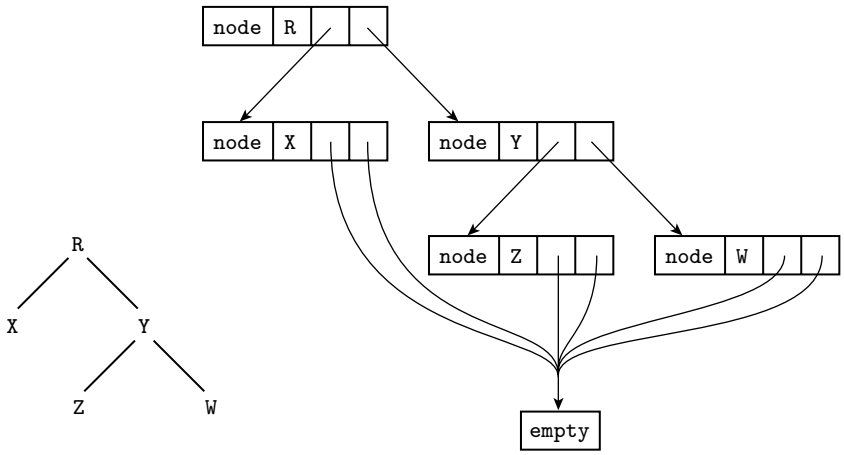


Figure 7.10 **Implementation of a tree in ML.** The abstract (conceptual) tree is shown at the lower left.

Reference Model

EXAMPLE 7.69
Tree type in ML

In ML, the `datatype` mechanism can be used to declare recursive types:

```
datatype chr_tree = empty | node of char * chr_tree * chr_tree;
```

Here a `chr_tree` is either an `empty` leaf or a `node` consisting of a character and two child trees. (Further details can be found in Section 7.2.4.)

It is natural in ML to include a `chr_tree` within a `chr_tree` because every variable is a reference. The tree `node (#"R", node (#"X", empty, empty), node (#"Y", node (#"Z", empty, empty), node (#"W", empty, empty)))` would most likely be represented in memory as shown in Figure 7.10. Each individual rectangle in the right-hand portion of this figure represents a block of storage allocated from the heap. In effect, the tree is a tuple (record) tagged to indicate that it is a `node`. This tuple in turn refers to two other tuples that are also tagged as `node`s. At the fringe of the tree are tuples that are tagged as `empty`; these contain no further references. Because all `empty` tuples are the same, the implementation is free to use just one, and to have every reference point to it. ■

EXAMPLE 7.70
Tree type in Lisp

In Lisp, which uses a reference model of variables but is not statically typed, our tree could be specified textually as `'(#\R (#\X ()()) (#\Y (#\Z ()()) (#\W ()())))`. Each level of parentheses brackets the elements of a *list*. In this case, the outermost such list contains three elements: the character `R` and nested lists to represent the left and right subtrees. (The prefix `#\` notation serves the same purpose as surrounding quotes in other languages.) Semantically, each list is a pair of references: one to the head and one to the remainder of the list. As we noted in Section 7.7.1, these semantics are almost always reflected in the implementation by a `cons` cell containing two pointers. A binary tree can thus be represented as

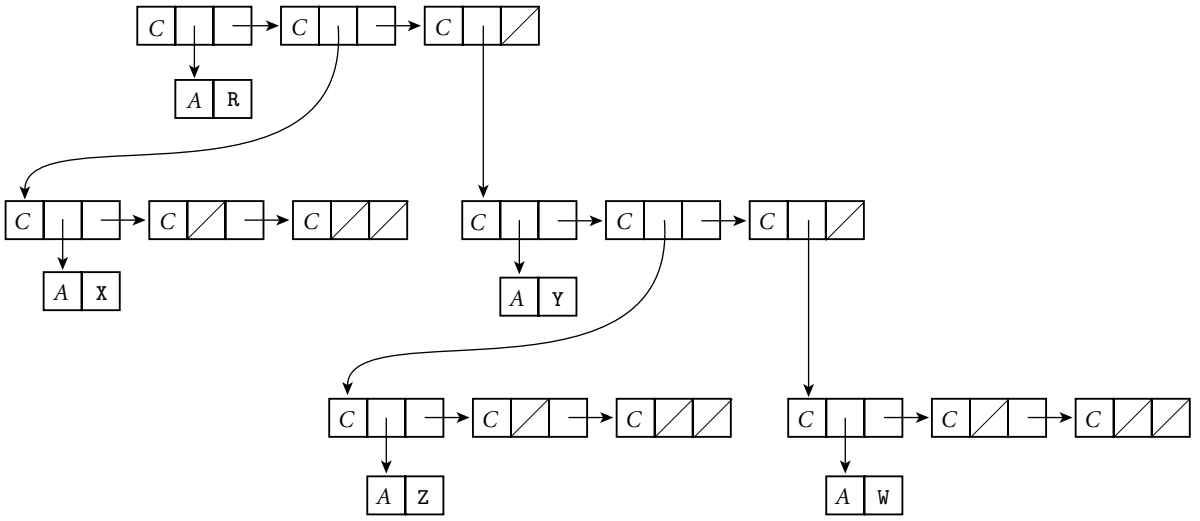


Figure 7.11 **Implementation of a tree in Lisp.** A diagonal slash through a box indicates a null pointer. The *C* and *A* tags serve to distinguish the two kinds of memory blocks: cons cells and blocks containing atoms.

a three-element (three `cons` cell) list, as shown in Figure 7.11. At the top level of the figure, the first `cons` cell points to `R`; the second and third point to nested lists representing the left and right subtrees. Each block of memory is tagged to indicate whether it is a `cons` cell or an *atom*. An atom is anything other than a `cons` cell; that is, an object of a built-in type (integer, real, character, string, etc.), or a user-defined structure (record) or array. The uniformity of Lisp lists (everything is a `cons` cell or an atom) makes it easy to write polymorphic functions, though without the static type checking of ML.

If one programs in a purely functional style in ML or in Lisp, the data structures created with recursive types turn out to be acyclic. New objects refer to old ones, but old ones never change, and thus never point to new ones. Circular structures can be defined only by using the imperative features of the languages. In ML, these features include an explicit notion of pointer, discussed briefly under "Value Model" below.

EXAMPLE 7.71
Mutually recursive types in ML

Even when writing in a functional style, one often finds a need for *types* that are *mutually recursive*. In a compiler, for example, it is likely that symbol table records and syntax tree nodes will need to refer to each other. A syntax tree node that represents a subroutine call will need to refer to the symbol table record that represents the subroutine. The symbol table record, for its part, will need to refer to the syntax tree node at the root of the subtree that represents the subroutine's code. If types are declared one at a time, and if names must be declared before they can be used, then whichever mutually recursive type is declared first will be unable to refer to the other. ML addresses this problem by allowing types to be declared together in a group:

```
datatype sym_tab_rec = variable of ...
    | type of ...
    | ...
    | subroutine of {code : syn_tree_node, ...}
and syn_tree_node = expression of ...
    | loop of ...
    | ...
    | subr_call of {subr : sym_tab_rec, ...};
```

Mutually recursive types of this sort are trivial in Lisp, since it is dynamically typed. (Common Lisp includes a notion of structures, but field types are not declared. In simpler Lisp dialects programmers use nested lists in which fields are merely positional conventions.) ■

Value Model

EXAMPLE 7.72
Tree types in Pascal, Ada, and C

In Pascal, our tree data type would be declared as follows:

```
type chr_tree_ptr = ^chr_tree;
     chr_tree = record
        left, right : chr_tree_ptr;
        val : char
     end;
```

The Ada declaration is similar:

```
type chr_tree;
type chr_tree_ptr is access chr_tree;
type chr_tree is record
    left, right : chr_tree_ptr;
    val : character;
end record;
```

In C, the equivalent declaration is

```
struct chr_tree {
    struct chr_tree *left, *right;
    char val;
};
```

As mentioned in Section 3.3.3, Pascal permits forward references in the declaration of pointer types, to support recursive types. Ada and C use incomplete type declarations instead. ■

EXAMPLE 7.73
Allocating heap nodes

No aggregate syntax is available for linked data structures in Pascal, Ada, or C; a tree must be constructed node by node. To allocate a new node from the heap, the programmer calls a built-in function. In Pascal:

```
new(my_ptr);
```

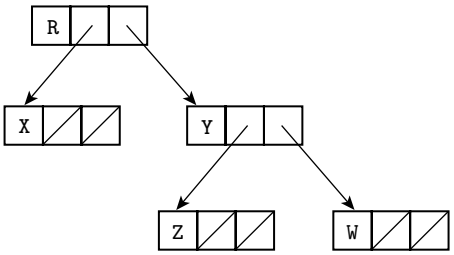


Figure 7.12 Typical implementation of a tree in a language with explicit pointers. As in Figure 7.11, a diagonal slash through a box indicates a null pointer.

In Ada:

```
my_ptr := new chr_tree;
```

In C:

```
my_ptr = malloc(sizeof(struct chr_tree));
```

C's `malloc` is defined as a library function, not a built-in part of the language (though some compilers recognize and optimize it as a special case). The programmer must specify the size of the allocated object explicitly, and while the return value (of type `void*`) can be assigned into any pointer, the assignment is not type-safe.

EXAMPLE 7.74
Object-oriented allocation of heap nodes

C++, Java, and C# replace `malloc` with a built-in, type-safe `new`:

```
my_ptr = new chr_tree( arg_list );
```

In addition to "knowing" the size of the requested type, the C++/Java/C# `new` will automatically call any user-specified *constructor* (initialization) function, passing the specified argument list. In a similar but less flexible vein, Ada's `new` may specify an initial value for the allocated object:

```
my_ptr := new chr_tree'(null, null, 'X');
```

EXAMPLE 7.75
Pointer-based tree

After we have allocated and linked together appropriate nodes in C, Pascal, or Ada, our tree example is likely to be implemented as shown in Figure 7.12. As in Lisp, a leaf is distinguished from an internal node simply by the fact that its two pointer fields are null.

EXAMPLE 7.76
Pointer dereferencing

To access the object referred to by a pointer, most languages use an explicit dereferencing operator. In Pascal and Modula this operator takes the form of a postfix "up-arrow":

```
my_ptr^.val := 'X';
```

In C it is a prefix star:

```
(*my_ptr).val = 'X';
```

Because pointers so often refer to records (`structs`), for which the prefix notation is awkward, C also provides a postfix "right-arrow" operator that plays the role of the "up-arrow dot" combination in Pascal:

```
my_ptr->val = 'X';
```

■

EXAMPLE 7.77
Implicit dereferencing in Ada

On the assumption that pointers almost always refer to records, Ada dispenses with dereferencing altogether. The same dot-based syntax can be used to access either a field of the record `foo` or a field of the record *pointed to* by `foo`, depending on the type of `foo`:

```
T : chr_tree;
P : chr_tree_ptr;
...
T.val := 'X';
P.val := 'Y';
```

In those cases in which one actually wants to name the *entire* object referred to by a pointer, Ada provides a special "pseudofield" called `all`:

```
T := P.all;
```

In essence, pointers in Ada are automatically dereferenced when needed. ■

EXAMPLE 7.78
Pointer dereferencing in ML

The imperative features of ML include an assignment statement, but this statement requires that the left-hand side be a pointer: its effect is to make the pointer refer to the object on the right-hand side. To access the object referred to by a pointer, one uses an exclamation point as a prefix dereferencing operator:

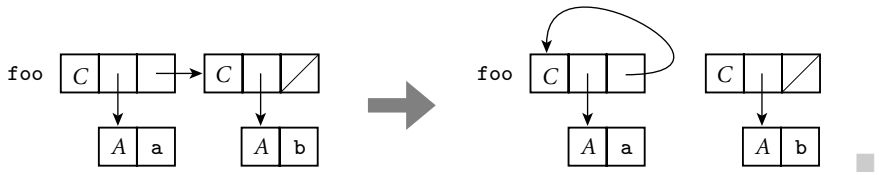
```
val p = ref 2;  (* p is a pointer to 2 *)
...
p := 3;         (* p now points to 3 *)
...
let val n = !p in ...
    (* n is simply 3 *)
```

ML thus makes the distinction between l-values and r-values very explicit. Most languages blur the distinction by implicitly dereferencing variables on the right-hand side of every assignment statement. Ada blurs the distinction further by dereferencing pointers automatically in certain circumstances. ■

The imperative features of Lisp do not include a dereferencing operator. Since every object has a self-evident type, and assignment is performed using a small set of built-in operators, there is never any ambiguity as to what is intended.

EXAMPLE 7.79
Assignment in Lisp

Assignment in Common Lisp employs the `setf` operator (Scheme uses `set!`, `set-car!`, and `set-cdr!`), rather than the more common `:=`. For example, if `foo` refers to a list, then `(cdr foo)` is the right-hand ("rest of list") pointer of the first node in the list, and the assignment `(set-cdr! foo foo)` makes this pointer refer back to `foo`, creating a one-node circular list:



Pointers and Arrays in C

EXAMPLE 7.80
Array names and pointers in C

Pointers and arrays are closely linked in C. Consider the following declarations:

```
int n;
int *a;          /* pointer to integer */
int b[10];       /* array of 10 integers */
```

Now all of the following are valid:

```
1. a = b;           /* make a point to the initial element of b */
2. n = a[3];
3. n = *(a+3);      /* equivalent to previous line */
4. n = b[3];
5. n = *(b+3);      /* equivalent to previous line */
```

In most contexts, an unsubscripted array name in C is automatically converted to a pointer to the array's first element (the one with index zero), as shown here in line 1. (Line 5 embodies the same conversion.) Lines 3 and 5 illustrate *pointer arithmetic*: Given a pointer to an element of an array, the addition of an integer k produces a pointer to the element k positions later in the array (earlier if k is negative.) The prefix `*` is a pointer dereference operator. Pointer arithmetic is valid only within the bounds of a single array, but C compilers are not required to check this.

Remarkably, the subscript operator `[]` in C is actually defined in terms of pointer arithmetic: lines 2 and 4 are syntactic sugar for lines 3 and 5, respectively. More precisely, `E1[E2]`, for any expressions `E1` and `E2`, is defined to be `(*((E1)+(E2)))`, which is of course the same as `(*((E2)+(E1)))`. (Extra parentheses have been used in this definition to avoid any questions of precedence if `E1` and `E2` are complicated expressions.) Correctness requires only that one operand of `[]` have an array or pointer type and the other have an integral type. Thus `A[3]` is equivalent to `3[A]`, something that comes as a surprise to most programmers.

EXAMPLE 7.81
Pointer comparison and subtraction in C

In addition to allowing an integer to be added to a pointer, C allows pointers to be subtracted from one another or compared for ordering, provided that they refer to elements of the same array. The comparison `p < q`, for example, tests to see if `p` refers to an element closer to the beginning of the array than the one referred to by `q`. The expression `p - q` returns the number of array positions that separate the elements to which `p` and `q` refer. All arithmetic operations on pointers "scale" their results as appropriate, based on the size of the referenced objects. For multidimensional arrays with row-pointer layout, `a[i][j]` is equivalent to `(*(a+i))[j]` or `*(a[i]+j)` or `*(*(a+i)+j)`. ■

EXAMPLE 7.82
Pointer and array declarations in C

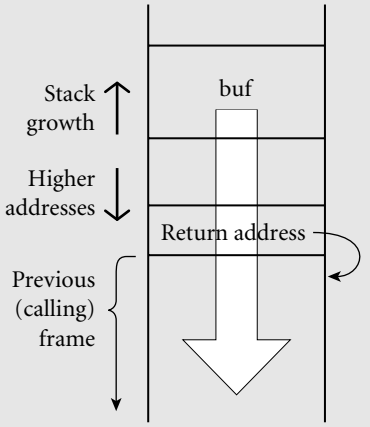
Despite the interoperability of pointers and arrays in C, programmers need to be aware that the two are not the same, particularly in the context of variable declarations, which need to allocate space when elaborated. The declaration of

DESIGN & IMPLEMENTATION

Stack smashing

The lack of bounds checking on array subscripts and pointer arithmetic is a major source of bugs and security problems in C. Many of the most infamous Internet viruses have propagated by means of *stack smashing*, a particularly nasty form of *buffer overflow attack*. Consider a (very naive) routine designed to read a number from an input stream:

```
int get_acct_num(FILE *s) {
    char buf[100];
    char *p = buf;
    do {
        /* read from stream s: */
        *p = getc(s);
    } while (*p++ != '\n');
    *p = '\0';
    /* convert ascii to int: */
    return atoi(buf);
}
```



If the stream provides more than 100 characters without a newline (`'\n'`), those characters will overwrite memory beyond the confines of `buf`, as shown by the large white arrow in the figure. A careful attacker may be able to invent a string whose bits include both a sequence of valid machine instructions and a replacement value for the subroutine's return address. When the routine attempts to return, it will jump into the attacker's instructions instead.

Stack smashing can be prevented by manually checking array bounds in C, or by configuring the hardware to prevent the execution of instructions in the stack (see the sidebar on page 179). It would never have been a problem in the first place, however, if C had been designed for automatic bounds checks.

a pointer variable allocates space to hold a pointer, while the declaration of an array variable allocates space to hold the whole array. In the case of an array the declaration must specify a size for each dimension. Thus `int *a[n]`, when elaborated, will allocate space for *n* row pointers; `int a[n][m]` will allocate space for a two-dimensional array with contiguous layout.[10] As a convenience, a variable declaration that includes initialization to an aggregate can omit the size of the outermost dimension if that information can be inferred from the contents of the aggregate:

```
int a[][2] = {{1, 2}, {3, 4}, {5, 6}};   // three rows
```

EXAMPLE 7.83
Arrays as parameters in C

When an array is included in the argument list of a function call, C passes a pointer to the first element of the array, not the array itself. For a one-dimensional array of integers, the corresponding formal parameter may be declared as `int a[]` or `int *a`. For a two-dimensional array of integers with row-pointer layout, the formal parameter may be declared as `int *a[]` or `int **a`. For a two-dimensional array with contiguous layout, the formal parameter may be declared as `int a[][m]` or `int (*a)[m]`. The size of the first dimension is irrelevant; all that is passed is a pointer, and C performs no dynamic checks to ensure that references are within the bounds of the array.

DESIGN & IMPLEMENTATION

Pointers and arrays

Many C programs use pointers instead of subscripts to iterate over the elements of arrays. Before the development of modern optimizing compilers, pointer-based array traversal often served to eliminate redundant address calculations, thereby leading to faster code. With modern compilers, however, the opposite may be true: redundant address calculations can be identified as common subexpressions, and certain other code improvements are easier for indices than they are for pointers. In particular, as we shall see in Chapter 16, pointers make it significantly more difficult for the code improver to determine when two l-values may be *aliases* for one other.

Today the use of pointer arithmetic is mainly a matter of personal taste: some C programmers consider pointer-based algorithms to be more elegant than their array-based counterparts; others simply find them harder to read. Certainly the fact that arrays are passed as pointers makes it natural to write subroutines in the pointer style.

10 To read declarations in C, it is helpful to follow the following rule: start at the name of the variable and work right as far as possible, subject to parentheses; then work left as far as possible; then jump out a level of parentheses and repeat. Thus `int *a[n]` means that `a` is an *n*-element array of pointers to integers, while `int (*a)[n]` means that `a` is a pointer to an *n*-element array of integers.

In all cases, a declaration must allow the compiler (or human reader) to determine the size of the *elements* of an array or, equivalently, the size of the objects referred to by a pointer. Thus neither `int a[][]` nor `int (*a)[]` is a valid variable or parameter declaration: neither provides the compiler with the size information it needs to generate code for `a + i` or `a[i]`.

EXAMPLE 7.84
`Sizeof` in C

The built-in `sizeof` operator returns the size in bytes of an object or type. When given an array as argument it returns the size of the entire array. When given a pointer as argument it returns the size of the pointer itself. If `a` is an array, `sizeof(a) / sizeof(a[0])` returns the number of elements in the array. Similarly, if pointers occupy 4 bytes and double-precision floating-point numbers occupy 8 bytes, then given

```
double *a;              /* pointer to double */
double (*b)[10];        /* pointer to array of 10 doubles */
```

we have `sizeof(a)` = `sizeof(b)` = 4, `sizeof(*a)` = `sizeof(*b[0])` = 8, and `sizeof(*b)` = 80. In most cases, `sizeof` can be evaluated at compile time. The principal exception occurs for variable-length arrays, whose size may not be known until elaboration time:

```
void f(int len) {
    int A[len];          /* sizeof(A) == len * sizeof(int) */
```

CHECK YOUR UNDERSTANDING

36. Name three languages that provide particularly extensive support for character strings.
37. Why might a language permit operations on strings that it does not provide for arrays?
38. What are the strengths and weaknesses of the bit-vector representation for sets? How else might sets be implemented?
39. Discuss the tradeoffs between pointers and the recursive types that arise naturally in a language with a reference model of variables.
40. Summarize the ways in which one dereferences a pointer in various programming languages.
41. What is the difference between a *pointer* and an *address*?
42. Discuss the advantages and disadvantages of the interoperability of pointers and arrays in C.
43. Under what circumstances must the bounds of a C array be specified in its declaration?

7.7.2 Dangling References

When a heap-allocated object is no longer live, a long-running program needs to reclaim the object's space. Stack objects are reclaimed automatically as part of the subroutine calling sequence. How are heap objects reclaimed? There are two alternatives. Languages like Pascal, C, and C++ require the programmer to reclaim an object explicitly. In Pascal:

EXAMPLE 7.85
Explicit storage reclamation

```
dispose(my_ptr);
```

In C:

```
free(my_ptr);
```

In C++:

```
delete my_ptr;
```

C++ provides additional functionality: prior to reclaiming the space, it automatically calls any user-provided *destructor* function for the object. A destructor can reclaim space for subsidiary objects, remove the object from indices or tables, print messages, or perform any other operation appropriate at the end of the object's lifetime.

A *dangling reference* is a live pointer that no longer points to a valid object. In languages like Algol 68 or C, which allow the programmer to create pointers to stack objects, a dangling reference may be created when a subroutine returns while some pointer in a wider scope still refers to a local object of that subroutine:

EXAMPLE 7.86
Dangling reference to a stack variable in C++

```
int i = 3;
int *p = &i;
...
void foo() { int n = 5;   p = &n; }
...
cout << *p;    // prints 3
foo();
...
cout << *p;    // undefined behavior: n is no longer live
```

In a language with explicit reclamation of heap objects, a dangling reference is created whenever the programmer reclaims an object to which pointers still refer:

EXAMPLE 7.87
Dangling reference to a heap variable in C++

```
int *p = new int;
*p = 3;
...
cout << *p;    // prints 3
delete p;
...
cout << *p;    // undefined behavior: *p has been reclaimed
```

Note that even if the reclamation operation were to change its argument to a null pointer, this would not solve the problem, because *other* pointers might still refer to the same object. ■

Because a language implementation may reuse the space of reclaimed stack and heap objects, a program that uses a dangling reference may read or write bits in memory that are now part of some other object. It may even modify bits that are now part of the implementation's bookkeeping information, corrupting the structure of the stack or heap.

Algol 68 addresses the problem of dangling references to stack objects by forbidding a pointer from pointing to any object whose lifetime is briefer than that of the pointer itself. Unfortunately, this rule is difficult to enforce. Among other things, since both pointers and objects to which pointers might refer can be passed as arguments to subroutines, dynamic semantic checks are possible only if reference parameters are accompanied by a hidden indication of lifetime. Ada 95 has a more restrictive rule that is easier to enforce: it forbids a pointer from pointing to any object whose lifetime is briefer than that of the pointer's *type*.

IN MORE DEPTH

On the PLP CD we consider two mechanisms that are sometimes used to catch dangling references at run time. *Tombstones* introduce an extra level of indirection on every pointer access. When an object is reclaimed, the indirection word (tombstone) is marked in a way that invalidates future references to the object. *Locks* and *keys* add a word to every pointer and to every object in the heap; these words must match for the pointer to be valid. Tombstones can be used in languages that permit pointers to nonheap objects, but they introduce the secondary problem of reclaiming the tombstones themselves. Locks and keys are somewhat simpler, but they work only for objects in the heap.

7.7.3 Garbage Collection

Explicit reclamation of heap objects is a serious burden on the programmer and a major source of bugs (memory leaks and dangling references). The code required to keep track of object lifetimes makes programs more difficult to design, implement, and maintain. An attractive alternative is to have the language implementation notice when objects are no longer useful and reclaim them automatically. Automatic reclamation (otherwise known as *garbage collection*) is more-or-less essential for functional languages: `delete` is a very imperative sort of operation, and the ability to construct and return arbitrary objects from functions means that many objects that would be allocated on the stack in an imperative language must be allocated from the heap in a functional language, to give them unlimited extent.

Over time, automatic garbage collection has become popular for imperative languages as well. It can be found in, among others, Clu, Cedar, Modula-3, Java,

C#, and all the major scripting languages. Automatic collection is difficult to implement, but the difficulty pales in comparison to the convenience enjoyed by programmers once the implementation exists. Automatic collection also tends to be slower than manual reclamation, though it eliminates any need to check for dangling references.

Reference Counts

When is an object no longer useful? One possible answer is: when no pointers to it exist.[11] The simplest garbage collection technique simply places a counter in each object that keeps track of the number of pointers that refer to the object. When the object is created, this *reference count* is set to 1, to represent the pointer returned by the `new` operation. When one pointer is assigned into another, the run-time system decrements the reference count of the object formerly referred to by the assignment's left-hand side, and increments the count of the object referred to by the right-hand side. On subroutine return, the calling sequence epilogue must decrement the reference count of any object referred to by a local pointer

DESIGN & IMPLEMENTATION

Garbage collection

Garbage collection presents a classic tradeoff between convenience and safety on the one hand and performance on the other. Manual storage reclamation, implemented correctly by the application program, is almost invariably faster than any automatic garbage collector. It is also more predictable: automatic collection is notorious for its tendency to introduce intermittent "hiccups" in the execution of real-time or interactive programs.

Ada takes the unusual position of refusing to take a stand: the language design makes automatic garbage collection possible, but implementations are not required to provide it, and programmers can request manual reclamation with a built-in routine called `Unchecked_Deallocation`. The Ada 95 version of the language provides extensive facilities whereby programmers can implement their own storage managers (garbage collected or not), with different types of pointers corresponding to different storage "pools."

In a similar vein, the Real Time Specification for Java allows the programmer to create so-called *scoped memory* areas that are accessible to only a subset of the currently running threads. When all threads with access to a given area terminate, the area is reclaimed in its entirety. Objects allocated in a scoped memory area are never examined by the garbage collector; performance anomalies due to garbage collection can therefore be avoided by providing scoped memory to every real-time thread.

---

11 Throughout the following discussion we will use the pointer-based terminology of languages with a value model of variables. The techniques apply equally well, however, to languages with a reference model of variables.

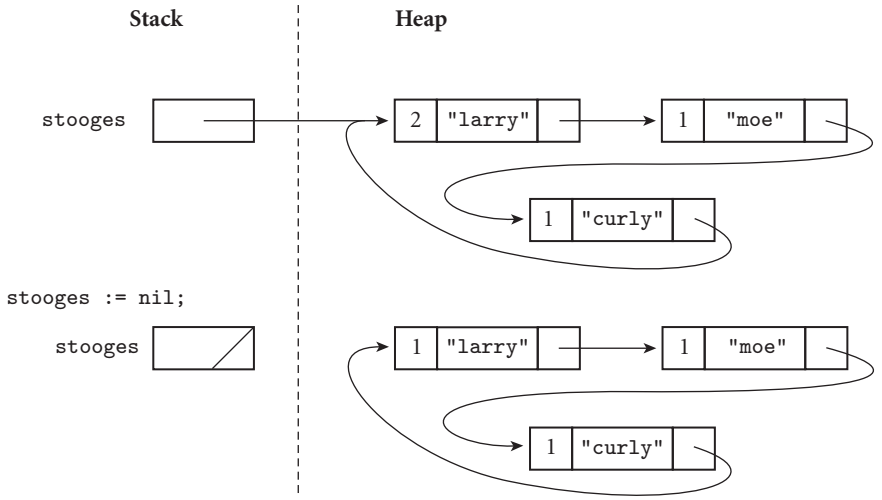


Figure 7.13 **Reference counts and circular lists.** The list shown here cannot be found via any program variable, but because it is circular, every cell contains a nonzero count.

that is about to be destroyed. When a reference count reaches zero, its object can be reclaimed. Recursively, the run-time system must decrement counts for any objects referred to by pointers within the object being reclaimed, and reclaim those objects if their counts reach zero. To prevent the collector from following garbage addresses, each pointer must be initialized to null at elaboration time.

In order for reference counts to work, the language implementation must be able to identify the location of every pointer. When a subroutine returns, it must be able to tell which words in the stack frame represent pointers; when an object in the heap is reclaimed, it must be able to tell which words within the object represent pointers. The standard technique to track this information relies on *type descriptors* generated by the compiler. There is one descriptor for every distinct type in the program, plus one for the stack frame of each subroutine, and one for the set of global variables. Most descriptors are simply a table that lists the offsets within the type at which pointers can be found, together with the addresses of descriptors for the types of the objects referred to by those pointers. For a tagged variant record (discriminated union) type, the descriptor is a bit more complicated: it must contain a list of values (or ranges) for the tag, together with a table for the corresponding variant. For *untagged* variant records, there is no acceptable solution: reference counts work only if the language is strongly typed (but see the discussion of "Conservative Collection" on page 364).

EXAMPLE 7.88
Reference counts and circular structures

The most important problem with reference counts stems from their definition of a "useful object." While it is definitely true that an object is useless if no references to it exist, it may also be useless when references *do* exist. As shown in Figure 7.13, reference counts may fail to collect circular structures. They work well only for structures that are guaranteed to be noncircular. Many language

implementations use reference counts for variable-length strings; strings never contain references to anything else. Perl uses reference counts for all dynamically allocated data; the manual warns the programmer to break cycles manually when data aren't needed anymore. Some purely functional languages may also be able to use reference counts safely in all cases, if the lack of an assignment statement prevents them from introducing circularity. Finally, reference counts can be used to reclaim tombstones. While it is certainly possible to create a circular structure with tombstones, the fact that the programmer is responsible for explicit deallocation of heap objects implies that reference counts will fail to reclaim tombstones only when the programmer has failed to reclaim the objects to which they refer. ■

Tracing Collection

A better definition of a "useful" object is one that can be reached by following a chain of valid pointers starting from something that has a name (i.e., something outside the heap). According to this definition, the blocks in the bottom half of Figure 7.13 are useless, even though their reference counts are nonzero. Tracing collectors work by recursively exploring the heap, starting from external pointers, to determine what is useful.

Mark-and-Sweep The classic mechanism to identify useless blocks, under this more accurate definition, is known as *mark-and-sweep*. It proceeds in three main steps, executed by the garbage collector when the amount of free space remaining in the heap falls below some minimum threshold.

DESIGN & IMPLEMENTATION

What exactly is garbage?

Reference counting implicitly defines a garbage object as one to which no pointers exist. Tracing implicitly defines it as an object that is no longer reachable from outside the heap. Ideally, we'd like an even stronger definition: a garbage object is one that the program will never use again. We settle for nonreachability because this ideal definition is uncomputable. The difference can matter in practice: if a program maintains a pointer to an object it will never use again, then the garbage collector will be unable to reclaim it. If the number of such objects grows with time, then the program has a memory leak, despite the presence of a garbage collector. (Trivially we could imagine a program that added every newly allocated object to a global list, but never actually perused the list. Such a program would defeat the collector entirely.)

For the sake of space efficiency, programmers are advised to "zero out" any pointers they no longer need. Doing this can be difficult, but not as difficult as fully manually reclamation—in particular, we do not need to realize when we are zeroing the *last* pointer to a given object. For the same reason, dangling references can never arise: the garbage collector will refrain from reclaiming any object that is reachable along some other path.

1. The collector walks through the heap, tentatively marking every block as "useless."
2. Beginning with all pointers outside the heap, the collector recursively explores all linked data structures in the program, marking each newly discovered block as "useful." (When it encounters a block that is already marked as "useful," the collector knows it has reached the block over some previous path, and returns without recursing.)
3. The collector again walks through the heap, moving every block that is still marked "useless" to the free list.

Several potential problems with this algorithm are immediately apparent. First, both the initial and final walks through the heap require that the collector be able to tell where every "in-use" block begins and ends. In a language with variable-size heap blocks, every block must begin with an indication of its size, and of whether it is currently free. Second, the collector must be able in Step 2 to find the pointers contained within each block. The standard solution is to place a pointer to a type descriptor near the beginning of each block.

Pointer Reversal The exploration step (Step 2) of mark-and-sweep collection is naturally recursive. The obvious implementation needs a stack whose maximum depth is proportional to the longest chain through the heap. In practice, the space for this stack may not be available: after all, we run garbage collection when we're about to run out of space![12] An alternative implementation of the exploration step uses a technique first suggested by Schorr and Waite [SW67] to embed the equivalent of the stack in already-existing fields in heap blocks. More specifically, as the collector explores the path to a given block, it *reverses* the pointers it follows, so that each points *back* to the previous block instead of forward to the next. This pointer-reversal technique is illustrated in Figure 7.14. As it explores, the collector keeps track of the current block and the block from whence it came.

EXAMPLE 7.89
Heap tracing with pointer reversal

To return from block X to block U (after part (d) of the figure), the collector will use the reversed pointer in U to restore its notion of previous block (T). It will then flip the reversed pointer back to X and update its notion of current block to U. If the block to which it has returned contains additional pointers, the collector will proceed forward again; otherwise it will return across the previous reversed pointer and try again. At most one pointer in every block will be reversed at any given time. This pointer must be marked, probably by means of another bookkeeping field at the beginning of each block. (We could mark the pointer by setting one of its low-order bits, but the cost in time would probably be prohibitive: we'd have to search the block on every visit.) ■

12 In many language implementations, the stack and heap grow toward each other from opposite ends of memory (Section 14.4); if the heap is full, the stack can't grow. In a system with virtual memory the distance between the two may theoretically be enormous, but the space that backs them up on disk is still limited, and shared between them.

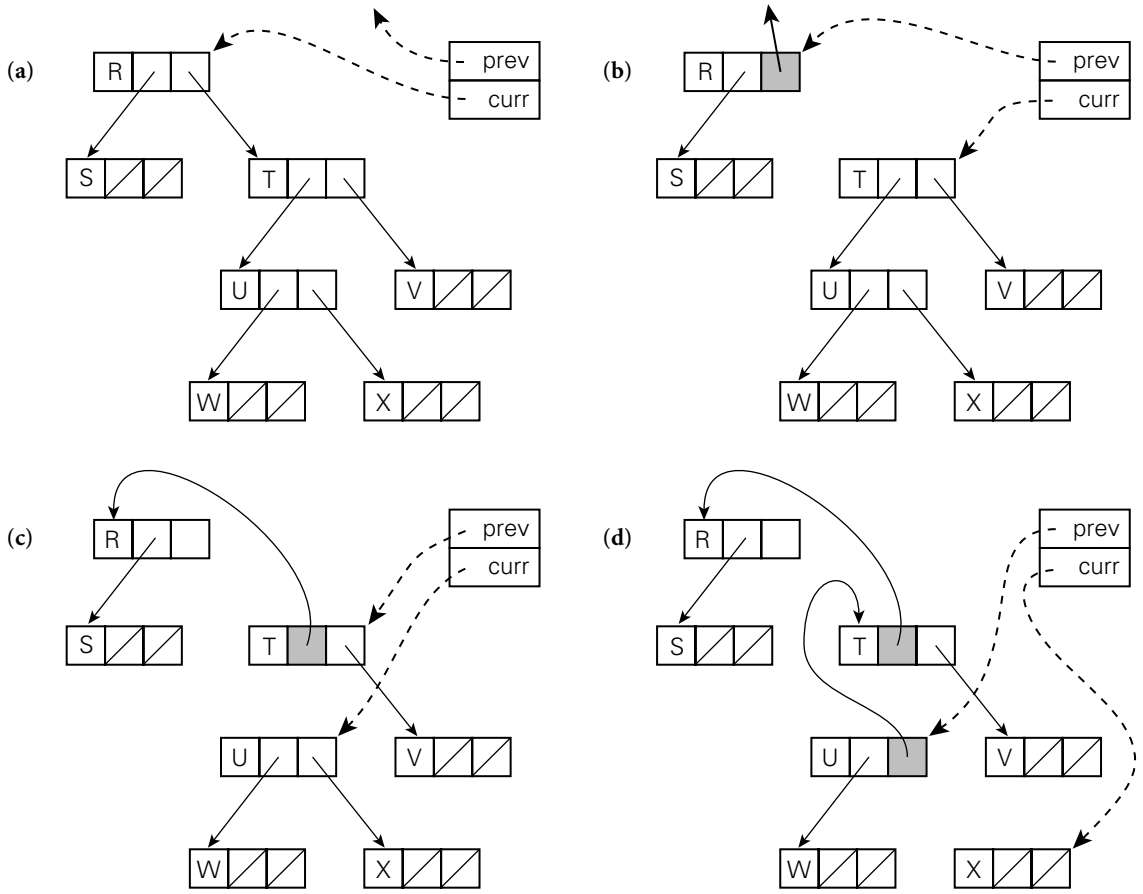


Figure 7.14 **Heap exploration via pointer reversal.** The block currently under examination is indicated by the curr pointer. The previous block is indicated by the prev pointer. As the garbage collector moves from one block to the next, it changes the pointer it follows to refer back to the previous block. When it returns to a block it restores the pointer. Each reversed pointer must be marked (indicated with a shaded box), to distinguish it from other, forward pointers in the same block.

Stop-and-Copy In a language with variable-size heap blocks, the garbage collector can reduce external fragmentation by performing storage compaction. Many garbage collectors employ a technique known as *stop-and-copy* that achieves compaction while simultaneously eliminating Steps 1 and 3 in the standard mark-and-sweep algorithm. Specifically, they divide the heap into two regions of equal size. All allocation happens in the first half. When this half is (nearly) full, the collector begins its exploration of reachable data structures. Each reachable block is copied into the second half of the heap, with no external fragmentation. The old version of the block, in the first half of the heap, is overwritten with a "useful" flag and a pointer to the new location. Any other pointer that refers to the same block (and is found later in the exploration) is set to point to the new location. When the

collector finishes its exploration, all useful objects have been moved (and compacted) into the second half of the heap, and nothing in the first half is needed anymore. The collector can therefore swap its notion of first and second halves, and the program can continue. Obviously, this algorithm suffers from the fact that only half of the heap can be used at any given time, but in a system with virtual memory it is only the virtual space that is underutilized; each "half" of the heap can occupy most of physical memory as needed. Moreover, by eliminating Steps 1 and 3 of standard mark-and-sweep, stop-and-copy incurs overhead proportional to the number of nongarbage blocks, rather than the total number of blocks.

Generational Collection To further reduce the cost of collection, some garbage collectors employ a "generational" technique, exploiting the observation that most dynamically allocated objects are short-lived. The heap is divided into multiple regions (often two). When space runs low the collector first examines the youngest region (the "nursery"), which it assumes is likely to have the highest proportion of garbage. Only if it is unable to reclaim sufficient space in this region does the collector examine the next-older region. To avoid leaking storage in long-running systems, the collector must be prepared, if necessary, to examine the entire heap. In most cases, however, the overhead of collection will be proportional to the size of the youngest region only.

Any object that survives some small number of collections (often one) in its current region is promoted (moved) to the next older region, in a manner reminiscent of stop-and-copy. Promotion requires, of course, that pointers from old objects

DESIGN & IMPLEMENTATION

Reference counts versus tracing

Reference counts require a counter field in every heap object. For small objects such as `cons` cells, this space overhead may be significant. The ongoing expense of updating reference counts when pointers are changed can also be significant in a program with large amounts of pointer manipulation. Other garbage collection techniques, however, have similar overheads. Tracing generally requires a reversed pointer indicator in every heap block, which reference counting does not, and generational collectors must generally incur overhead on every pointer assignment in order to keep track of pointers into the newest section of the heap.

The two principal tradeoffs between reference counting and tracing are the inability of the former to handle cycles and the tendency of the latter to "stop the world" periodically in order to reclaim space. On the whole, implementors tend to favor reference counting for applications in which circularity is not an issue, and tracing collectors in the general case. The "stop the world" problem can be addressed with *incremental* or *concurrent* collectors, which interleave their execution with the rest of the program, but these tend to have higher total overhead. Efficient, effective garbage collection techniques remain an active area of research.

to new objects be updated to reflect the new locations. While such old-space-to-new-space pointers tend to be rare, a generational collector must be able to find them all quickly. At each pointer assignment, the compiler generates code to check whether the new value is an old-to-new pointer; if so, it adds the pointer to a hidden list accessible to the collector. This instrumentation on assignments is known as a *write barrier*.[13]

Conservative Collection Language implementors have traditionally assumed that automatic storage reclamation is possible only in languages that are strongly typed: both reference counts and tracing collection require that we be able to find the pointers within an object. If we are willing to admit the possibility that some garbage will go unreclaimed, it turns out that we can implement mark-and-sweep collection without being able to find pointers [BW88]. The key is to observe that any given block in the heap spans a relatively small number of addresses. There is only a very small probability that some word in memory that is not a pointer will happen to contain a bit pattern that looks like one of those addresses.

If we assume, conservatively, that everything that seems to point into a heap block is in fact a valid pointer, then we can proceed with mark-and-sweep collection. When space runs low, the collector (as usual) tentatively marks all blocks in the heap as useless. It then scans all word-aligned quantities in the stack and in global storage. If any of these words appears to contain the address of something in the heap, the collector marks the block that contains that address as useful. Recursively, the collector then scans all word-aligned quantities in the block, and marks as useful any other blocks whose addresses are found therein. Finally (as usual), the collector reclaims any blocks that are still marked useless.

The algorithm is completely safe (in the sense that it never reclaims useful blocks) so long as the programmer never "hides" a pointer. In C, for example, the collector is unlikely to function correctly if the programmer casts a pointer to `int` and then `xor`s it with a constant, with the expectation of restoring and using the pointer at a later time. In addition to sometimes leaving garbage unclaimed, conservative collection suffers from the inability to perform compaction: the collector can never be sure which "pointers" should be changed.

7.8 Lists

A list is defined recursively as either the empty list or a pair consisting of an object (which may be either a list or an atom) and another (shorter) list. Lists are ideally suited to programming in functional and logic languages, which do most of their work via recursion and higher-order functions (to be described in Section 10.5).

---

13 Unfortunately, the word "barrier" is heavily overloaded. Garbage collection barriers are unrelated to the synchronization barriers of Section 12.3.1, the memory barriers of Section 12.3.3, or the RTL barriers of Section ©14.2.2.

In Lisp, in fact, a program *is* a list, and can extend itself at run time by constructing a list and executing it (this capability will be examined further in Section 10.3.5; it depends heavily on the fact that Lisp delays almost all semantic checking until run time).

Lists can also be used in imperative programs. Clu provides a built-in type constructor for lists, and a list class is easy to write in most object-oriented languages. Most scripting languages provide extensive list support. In any language with records and pointers, the programmer can build lists by hand. Since many of the standard list operations tend to generate garbage, lists work best in a language with automatic garbage collection.

We have already discussed certain aspects of lists in ML (Section 7.2.4) and Lisp (Section 7.7.1). As we noted in those sections, lists in ML are homogeneous: every element of the list must have the same type. Lisp lists, by contrast, are heterogeneous: any object may be placed in a list, so long as it is never used in an inconsistent fashion.[14] The different approaches to type in ML and in Lisp lead to different implementations. An ML list is usually a chain of blocks, each of which contains an element and a pointer to the next block. A Lisp list is a chain of `cons` cells, each of which contains *two* pointers, one to the element and one to the next `cons` cell (see Figures 7.10 and 7.11, pages 347 and 348). For historical reasons, the two pointers in a `cons` cell are known as the `car` and the `cdr`; they represent the head of the list and the remaining elements, respectively. In both semantics (homogeneity vs heterogeneity) and implementation (chained blocks vs `cons` cells), Clu resembles ML, while Python and Prolog (to be discussed in Section 11.2) resemble Lisp. ■

EXAMPLE 7.90
Lists in ML and Lisp

EXAMPLE 7.91
List notation

Both ML and Lisp provide convenient notation for lists. An ML list is enclosed in square brackets, with elements separated by commas: `[a, b, c, d]`. A Lisp list is enclosed in parentheses, with elements separated by white space: `(a b c d)`. In both cases, the notation represents a *proper* list—one whose innermost pair consists of the final element and the empty list. In Lisp, it is also possible to construct an *improper* list, whose final pair contains two elements. (Strictly speaking, such a list does not conform to the standard recursive definition.) Lisp systems provide a more general, but cumbersome *dotted* list notation that captures both proper and improper lists. A dotted list is either an atom (possibly null) or a pair consisting of two dotted lists separated by a period and enclosed in parentheses. The dotted list `(a . (b . (c . (d . null))))` is the same as `(a b c d)`. The list `(a . (b . (c . d)))` is improper; its final `cons` cell contains a pointer to `d` in the second position, where a pointer to a list is normally required. ■

Both ML and Lisp provide a wealth of built-in polymorphic functions to manipulate arbitrary lists. Because programs are lists in Lisp, Lisp must distinguish between lists that are to be evaluated and lists that are to be left "as is," as structures. To prevent a literal list from being evaluated, the Lisp programmer may

14 Recall that objects are self-descriptive in Lisp. The only type checking occurs when a function "deliberately" inspects an argument to see whether it is a list or an atom of some particular type.

quote it: `(quote (a b c d))`, abbreviated `'(a b c d)`. To evaluate an internal list (e.g., one returned by a function), the programmer may pass it to the built-in function `eval`. In ML, programs are not lists, so a literal list is always a structural aggregate.

EXAMPLE 7.92
Basic list operations in Lisp

The most fundamental operations on lists are those that construct them from their components or extract their components from them. In Lisp:

```
(cons 'a '(b))          ⟹ (a b)
(car '(a b))            ⟹ a
(car nil)               ⟹ ??
(cdr '(a b c))          ⟹ (b c)
(cdr '(a))              ⟹ nil
(cdr nil)               ⟹ ??
(append '(a b) '(c d))  ⟹ (a b c d)
```

Here we have used ⟹ to mean "evaluates to." The `car` and `cdr` of the empty list (`nil`) are defined to be `nil` in Common Lisp; in Scheme they result in a dynamic semantic error. ■

EXAMPLE 7.93
Basic list operations in ML

In ML the equivalent operations are written as follows:

```
a :: [b]          ⟹ [a, b]
hd [a, b]         ⟹ a
hd []             ⟹ run-time exception
tl [a, b, c]      ⟹ [b, c]
tl [a]            ⟹ nil
tl []             ⟹ run-time exception
[a, b] @ [c, d]   ⟹ [a, b, c, d]
```

Run-time exceptions may be *caught* by the program if desired; further details will appear in Section 8.5. ■

DESIGN & IMPLEMENTATION

`Car` and `cdr`

The names of the functions `car` and `cdr` are historical accidents: they derive from the original (1959) implementation of Lisp on the IBM 704 at MIT. The machine architecture included 15-bit "address" and "decrement" fields in some of the (36-bit) loop-control instructions, together with additional instructions to load an index register from, or store it to, one of these fields within a 36-bit memory word. The designers of the Lisp interpreter decided to make `cons` cells mimic the internal format of instructions, so they could exploit these special instructions. In now archaic usage, memory words were also known as "registers." What might appropriately have been called "first" and "rest" pointers thus came to be known as the `CAR` (contents of address of register) and `CDR` (contents of decrement of register). The 704, incidentally, was also the machine on which Fortran was first developed, and the first commercial machine to include hardware floating-point and magnetic core memory.

Both ML and Lisp provide many additional list functions, including ones that test a list to see if it is empty; return the length of a list; return the *n*th element of a list, or a list consisting of all but the first *n* elements; reverse the order of the elements of a list; search a list for elements matching some predicate; or apply a function to every element of a list, returning the results as a list.

Miranda, Haskell, Python, and F# provide lists that resemble those of ML, but with an important additional mechanism, known as *list comprehensions*. These are adapted from traditional mathematical set notation. A common form comprises an expression, an enumerator, and one or more filters. In Haskell, the following denotes a list of the squares of all odd numbers less than 100:

EXAMPLE 7.94
List comprehensions

```
[i*i | i <- [1..100], i `mod` 2 == 1]
```

In Python we would write

```
[i*i for i in range(1, 100) if i % 2 == 1]
```

In F# the equivalent is

```
[for i in 1..100 do if i % 2 = 1 then yield i*i]
```

All of these are the equivalent of the mathematical

$$\{i \times i \mid i \in \{1, \ldots, 100\} \wedge i \bmod 2 = 1\}$$

We could of course create an equivalent list with a series of appropriate function calls. The brevity of the list comprehension syntax, however, can sometimes lead to remarkably elegant programs (see, e.g., Exercise 7.26). ■

7.9 Files and Input/Output

Input/output (I/O) facilities allow a program to communicate with the outside world. In discussing this communication, it is customary to distinguish between *interactive* I/O and I/O with files. Interactive I/O generally implies communication with human users or physical devices, which work in parallel with the running program, and whose input to the program may depend on earlier output from the program (e.g., prompts). Files generally refer to off-line storage implemented by the operating system. Files may be further categorized into those that are *temporary* and those that are *persistent*. Temporary files exist for the duration of a single program run; their purpose is to store information that is too large to fit in the memory available to the program. Persistent files allow a program to read data that existed before the program began running, and to write data that will continue to exist after the program has ended.

I/O is one of the most difficult aspects of a language to design, and one that displays the least commonality from one language to the next. Some languages provide built-in `file` data types and special syntactic constructs for I/O. Others relegate I/O entirely to library packages, which export a (usually opaque) `file` type and a variety of input and output subroutines. The principal advantage of language integration is the ability to employ non–subroutine-call syntax, and to perform operations (e.g., type checking on subroutine calls with varying numbers of parameters) that may not otherwise be available to library routines. A purely library-based approach to I/O, on the other hand, may keep a substantial amount of "clutter" out of the language definition.

IN MORE DEPTH

An overview of language-level I/O mechanisms can be found on the PLP CD. After a brief introduction to interactive and file-based I/O, we focus mainly on the common case of *text files*. The data in a text file are stored in character form, but may be converted to and from internal types during `read` and `write` operations. As examples, we consider the text I/O facilities of Fortran, Ada, C, and C++.

7.10 Equality Testing and Assignment

For simple, primitive data types such as integers, floating-point numbers, or characters, equality testing and assignment are relatively straightforward operations, with obvious semantics and obvious implementations (bit-wise comparison or copy). For more complicated or abstract data types, however, both semantic and implementation subtleties arise.

Consider for example the problem of comparing two character strings. Should the expression `s = t` determine whether `s` and `t`

- are aliases for one another?
- occupy storage that is bit-wise identical over its full length?
- contain the same sequence of characters?
- would appear the same if printed?

The second of these tests is probably too low-level to be of interest in most programs; it suggests the possibility that a comparison might fail because of garbage in currently unused portions of the space reserved for a string. The other three alternatives may all be of interest in certain circumstances, and may generate different results.

In many cases the definition of equality boils down to the distinction between l-values and r-values: in the presence of references, should expressions be considered equal only if they refer to the same object, or also if the objects to which

they refer are in some sense equal? The first option (refer to the same object) is known as a *shallow* comparison. The second (refer to equal objects) is called a *deep* comparison. For complicated data structures (e.g., lists or graphs) a deep comparison may require recursive traversal.

In imperative programming languages, assignment operations may also be deep or shallow. Under a reference model of variables, a shallow assignment `a := b` will make `a` refer to the object to which `b` refers. A deep assignment will create a copy of the object to which `b` refers, and make `a` refer to the copy. Under a value model of variables, a shallow assignment will copy the value of `b` into `a`, but if that value is a pointer (or a record containing pointers), then the objects to which the pointer(s) refer will not be copied.

EXAMPLE 7.95
Equality testing in Scheme

Most programming languages employ both shallow comparisons and shallow assignment. A few (notably Python and the various dialects of Lisp) provide more than one option for comparison. Scheme, for example, has three general-purpose equality-testing functions:

```
(eq? a b)        ; do a and b refer to the same object?
(eqv? a b)       ; are a and b known to be semantically equivalent?
(equal? a b)     ; do a and b have the same recursive structure?
```

Both `eq?` and `eqv?` perform a shallow comparison. The former may be faster for certain types in certain implementations; in particular, `eqv?` is required to detect the equality of values of the same discrete type, stored in different locations; `eq?` is not. The simpler `eq?` behaves as one would expect for Booleans, symbols (names), and pairs (things built by `cons`), but can have implementation-defined behavior on numbers, characters, and strings:

```
(eq? #t #t)            ⟹  #t (true)
(eq? 'foo 'foo)        ⟹  #t
(eq? '(a b) '(a b))    ⟹  #f (false); created by separate cons-es
(let ((p '(a b)))
  (eq? p p))           ⟹  #t; created by the same cons
(eq? 2 2)              ⟹  implementation dependent
(eq? "foo" "foo")      ⟹  implementation dependent
```

In any particular implementation, numeric, character, and string tests will always work the same way; if `(eq? 2 2)` returns `true`, then `(eq? 37 37)` will return `true` also. Implementations are free to choose whichever behavior results in the fastest code.

The exact rules that govern the situations in which `eqv?` is guaranteed to return `true` or `false` are quite involved. Among other things, they specify that `eqv?` should behave as one might expect for numbers, characters, and nonempty strings, and that two objects will never test `true` for `eqv?` if there are any circumstances under which they would behave differently. (Conversely, however, `eqv?` is allowed to return `false` for certain objects—functions, for example—that would behave

identically in all circumstances.)[15] The `eqv?` predicate is "less discriminating" than `eq?`, in the sense that `eqv?` will never return `false` when `eq?` returns `true`.

For structures (lists), `eqv?` returns `false` if its arguments refer to different root `cons` cells. In many programs this is not the desired behavior. The `equal?` predicate recursively traverses two lists to see if their internal structure is the same and their leaves are `eqv?`. The `equal?` predicate may lead to an infinite loop if the programmer has used the imperative features of Scheme to create a circular list.

Deep assignments are relatively rare. They are used primarily in distributed computing, and in particular for parameter passing in remote procedure call (RPC) systems. These will be discussed in Section 12.5.4.

For user-defined abstractions, no single language-specified mechanism for equality testing or assignment is likely to produce the desired results in all cases. Languages with sophisticated data abstraction mechanisms usually allow the programmer to define the comparison and assignment operators for each new data type—or to specify that equality testing and/or assignment is not allowed.

CHECK YOUR UNDERSTANDING

44. What are *dangling references*? How are they created, and why are they a problem?
45. What is *garbage*? How is it created, and why is it a problem? Discuss the comparative advantages of *reference counts* and *tracing collection* as a means of solving the problem.
46. Summarize the differences among mark-and-sweep, stop-and-copy, and generational garbage collection.
47. What is *pointer reversal*? What problem does it address?
48. What is "conservative" garbage collection? How does it work?
49. Do dangling references and garbage ever arise in the same programming language? Why or why not?
50. Why was automatic garbage collection so slow to be adopted by imperative programming languages?
51. What are the advantages and disadvantages of allowing pointers to refer to objects that do not lie in the heap?
52. Why are lists so heavily used in functional programming languages?
53. Why is equality testing more subtle than it first appears?

---

15 Significantly, `eqv?` is also allowed to return false when comparing numeric values of different types: `(eqv? 1 1.0)` may evaluate to `#f`. For numeric code, one generally wants the separate `=` function: `(= val1 val2)` will perform the necessary coercion and test for numeric equality (subject to rounding errors).

7.11 Summary and Concluding Remarks

This section concludes the third of our five core chapters on language design (names [from Part I], control flow, types, subroutines, and classes). In the first two sections we looked at the general issues of type systems and type checking. In the remaining sections we examined the most important composite types: records and variants, arrays and strings, sets, pointers and recursive types, lists, and files. We noted that types serve two principal purposes: they provide implicit context for many operations, freeing the programmer from the need to specify that context explicitly, and they allow the compiler to catch a wide variety of common programming errors. A *type system* consists of a set of built-in types, a mechanism to define new types, and rules for *type equivalence*, *type compatibility*, and *type inference*. Type equivalence determines when two names or values have the same type. Type compatibility determines when a value of one type may be used in a context that "expects" another type. Type inference determines the type of an expression based on the types of its components or (sometimes) the surrounding context. A language is said to be *strongly typed* if it never allows an operation to be applied to an object that does not support it; a language is said to be *statically typed* if it enforces strong typing at compile time.

In our general discussion of types we distinguished between the denotational, constructive, and abstraction-based points of view, which regard types, respectively, in terms of their values, their substructure, and the operations they support. We introduced terminology for the common built-in types and for enumerations, subranges, and the common type *constructors*. We discussed several different approaches to type equivalence, compatibility, and inference, including (on the PLP CD) a detailed examination of the inference rules of ML. We also examined type *conversion*, *coercion*, and *nonconverting casts*. In the area of type equivalence, we contrasted the *structural* and *name*-based approaches, noting that while name equivalence appears to have gained in popularity, structural equivalence retains its advocates.

In our survey of composite types, we spent the most time on records, arrays, and recursive types. Key issues for records include the syntax and semantics of variant records, whole-record operations, type safety, and the interaction of each of these with memory layout. Memory layout is also important for arrays, in which it interacts with binding time for shape; static, stack, and heap-based allocation strategies; efficient array traversal in numeric applications; the interoperability of pointers and arrays in C; and the available set of whole-array and *slice*-based operations.

For recursive data types, much depends on the choice between the *value* and *reference models* of variables/names. Recursive types are a natural fallout of the reference model; with the value model they require the notion of a *pointer*: a variable whose value is a reference. The distinction between values and references is important from an implementation point of view: it would be wasteful to implement built-in types as references, so languages with a reference model generally

implement built-in and user-defined types differently. Java reflects this distinction in the language semantics, calling for a value model of built-in types and a reference model for objects of user-defined class types.

Recursive types are generally used to create linked data structures. In most cases these structures must be allocated from a heap. In some languages, the programmer is responsible for deallocating heap objects that are no longer needed. In other languages, the language run-time system identifies and reclaims such *garbage* automatically. Explicit deallocation is a burden on the programmer, and leads to the problems of *memory leaks* and *dangling references*. While language implementations almost never attempt to catch memory leaks (see Exploration 3.32 and Exercise ◎7.36, however, for some ideas on this subject) *tombstones* or *locks* and *keys* are sometimes used to catch dangling references. Automatic garbage collection can be expensive, but has proven increasingly popular. Most garbage-collection techniques rely either on *reference counts* or on some form of recursive exploration (*tracing*) of currently accessible structures. Techniques in this latter category include *mark-and-sweep*, *stop-and-copy*, and *generational* collection.

Few areas of language design display as much variation as I/O. Our discussion (largely on the PLP CD) distinguished between *interactive I/O*, which tends to be very platform specific, and *file-based I/O*, which subdivides into *temporary files*, used for voluminous data within a single program run, and *persistent files*, used for off-line storage. Files also subdivide into those that represent their information in a binary form that mimics layout in memory and those that convert to and from character-based *text*. In comparison to binary files, text files generally incur both time and space overhead, but they have the important advantages of portability and human readability.

In our examination of types, we saw many examples of language innovations that have served to improve the clarity and maintainability of programs, often with little or no performance overhead. Examples include the original idea of user-defined types (Algol 68), enumeration and subrange types (Pascal), the integration of records and variants (Pascal), and the distinction between subtypes and derived types in Ada. In Chapter 9 we will examine what many consider the most important innovation of the past 30 years, namely object orientation.

In some cases, the distinctions between languages are less a matter of evolution than of fundamental differences in philosophy. We have already mentioned the choice between the value and reference models of variables/names. In a similar vein, most languages have adopted static typing, but Smalltalk, Lisp, and the many scripting languages work well with dynamic types. Most statically typed languages have adopted name equivalence, but ML and Modula-3 work well with structural equivalence. Most languages have moved away from type coercions, but C++ embraces them: together with operator overloading, they make it possible to define terse, type-safe I/O routines outside the language proper.

As in the previous chapter, we saw several cases in which a language's convenience, orthogonality, or type safety appears to have been compromised in order to simplify the compiler, or to make compiled programs smaller or faster. Examples include the lack of an equality test for records in most languages, the requirement

in Pascal and Ada that the variant portion of a record lie at the end, the limitations in many languages on the maximum size of sets, the lack of type checking for I/O in C, and the general lack of dynamic semantic checks in many language implementations. We also saw several examples of language features introduced at least in part for the sake of efficient implementation. These include `packed` types, multi-length numeric types, `with` statements, decimal arithmetic, and C-style pointer arithmetic.

At the same time, one can identify a growing willingness on the part of language designers and users to tolerate complexity and cost in language implementation in order to improve semantics. Examples here include the type-safe variant records of Ada; the standard-length numeric types of Java and C#; the variable-length strings and string operators of Icon, Java, and C#; the late binding of array bounds in Ada; and the wealth of whole-array and slice-based array operations in Fortran 90. One might also include the polymorphic type inference of ML. Certainly one should include the trend toward automatic garbage collection. Once considered too expensive for production-quality imperative languages, garbage collection is now standard not only in such experimental languages as Clu and Cedar, but in Ada, Modula-3, Java, and C# as well. Many of these features, including variable-length strings, slices, and garbage collection, have been embraced by scripting languages.

7.12 Exercises

7.1 Most statically typed languages developed since the 1970s (including Java, C#, and the descendants of Pascal) use some form of name equivalence for types. Is structural equivalence a bad idea? Why or why not?

7.2 In the following code, which of the variables will a compiler consider to have compatible types under structural equivalence? Under strict name equivalence? Under loose name equivalence?

```
type  T = array [1..10] of integer
      S = T
A  : T
B  : T
C  : S
D  : array [1..10] of integer
```

7.3 Consider the following declarations:

```
1.  type cell              – – a forward declaration
2.  type cell_ptr = pointer to cell
3.  x : cell
4.  type cell = record
5.        val : integer
6.        next : cell_ptr
7.  y : cell
```

Should the declaration at line 4 be said to introduce an alias type? Under strict name equivalence, should `x` and `y` have the same type? Explain.

7.4 Suppose you are implementing an Ada compiler, and must support arithmetic on 32-bit fixed-point binary numbers with a programmer-specified number of fractional bits. Describe the code you would need to generate to add, subtract, multiply, or divide two fixed-point numbers. You should assume that the hardware provides arithmetic instructions only for integers and IEEE floating-point. You may assume that the integer instructions preserve full precision; in particular, integer multiplication produces a 64-bit result. Your description should be general enough to deal with operands and results that have different numbers of fractional bits.

7.5 When Sun Microsystems ported Berkeley Unix from the Digital VAX to the Motorola 680x0 in the early 1980s, many C programs stopped working, and had to be repaired. In effect, the 680x0 revealed certain classes of program bugs that one could "get away with" on the VAX. One of these classes of bugs occurred in programs that use more than one size of integer (e.g., `short` and `long`), and arose from the fact that the VAX is a little-endian machine, while the 680x0 is big-endian (Section ©5.2). Another class of bugs occurred in programs that manipulate both null and empty strings. It arose from the fact that location zero in a Unix process's address space on the VAX always contained a zero, while the same location on the 680x0 is not in the address space, and will generate a protection error if used. For both of these classes of bugs, give examples of program fragments that would work on a VAX but not on a 680x0.

7.6 Ada provides two "remainder" operators, `rem` and `mod` for integer types, defined as follows [Ame83, Sec. 4.5.5]:

> Integer division and remainder are defined by the relation `A = (A/B)*B + (A rem B)`, where `(A rem B)` has the sign of `A` and an absolute value less than the absolute value of `B`. Integer division satisfies the identity `(-A)/B = -(A/B) = A/(-B)`.
>
> The result of the modulus operation is such that `(A mod B)` has the sign of `B` and an absolute value less than the absolute value of `B`; in addition, for some integer value `N`, this result must satisfy the relation `A = B*N + (A mod B)`.

Give values of `A` and `B` for which `A rem B` and `A mod B` differ. For what purposes would one operation be more useful than the other? Does it make sense to provide both, or is it overkill?

Consider also the `%` operator of C and the `mod` operator of Pascal. The designers of these languages could have picked semantics resembling those of either Ada's `rem` or its `mod`. Which did they pick? Do you think they made the right choice?

7.7 Consider the problem of performing range checks on set expressions in Pascal. Given that a set may contain many elements, some of which may be known at compile time, describe the information that a compiler might maintain in order to track both the elements known to belong to the set and the possible range of unknown elements. Then explain how to update

this information for the following set operations: union, intersection, and difference. The goal is to determine (1) when subrange checks can be eliminated at run time and (2) when subrange errors can be reported at compile time. Bear in mind that the compiler *cannot* do a perfect job: some unnecessary run-time checks will inevitably be performed, and some operations that must always result in errors will not be caught at compile time. The goal is to do as good a job as possible at reasonable cost.

7.8 Suppose we are compiling for a machine with 1-byte characters, 2-byte shorts, 4-byte integers, and 8-byte reals, and with alignment rules that require the address of every primitive data element to be an even multiple of the element's size. Suppose further that the compiler is not permitted to reorder fields. How much space will be consumed by the following array? Explain.

```
A : array [0..9] of record
        s : short
        c : char
        t : short
        d : char
        r : real
        i : integer
```

7.9 In Example 7.45 we suggested the possibility of sorting record fields by their alignment requirement, to minimize holes. In the example, we sorted smallest-alignment-first. What would happen if we sorted longest-alignment-first? Do you see any advantages to this scheme? Any disadvantages? If the record as a whole must be an even multiple of the longest alignment, do the two approaches ever differ in total space required?

7.10 Give Ada code to map from lowercase to uppercase letters, using

(a) an array

(b) a function

Note the similarity of syntax: in both cases `upper('a')` is `'A'`.

7.11 In Section 7.4.2 we noted that in a language with dynamic arrays and a value model of variables, records could have fields whose size is not known at compile time. To accommodate these, we suggested using a dope vector for the record, to track the offsets of the fields.

Suppose instead that we want to maintain a static offset for each field. Can we devise an alternative strategy inspired by the stack frame layout of Figure 7.6, and divide each record into a fixed-size part and a variable-size part? What problems would we need to address? (Hint: consider nested records.)

7.12 Explain how to extend Figure 7.6 to accommodate subroutine arguments that are passed by value, but whose shape is not known until the subroutine is called at run time.

7.13 Explain how to obtain the effect of Fortran 90's `allocate` statement for one-dimensional arrays using pointers in C. You will probably find that your solution does not generalize to multidimensional arrays. Why not? If you are familiar with C++, show how to use its `class` facilities to solve the problem.

7.14 In Section 7.4.3 we discussed how to differentiate between the constant and variable portions of an array reference, in order to efficiently access the subparts of array and record objects. An alternative approach is to generate naive code and count on the compiler's code improver to find the constant portions, group them together, and calculate them at compile time. Discuss the advantages and disadvantages of each approach.

7.15 Consider the following C declaration, compiled on a 32-bit Pentium machine:

```
struct {
    int n;
    char c;
} A[10][10];
```

If the address of `A[0][0]` is 1000 (decimal), what is the address of `A[3][7]`?

7.16 Suppose we are generating code for a Pascal-like language on a RISC machine with the following characteristics: 8-byte floating-point numbers, 4-byte integers, 1-byte characters, and 4-byte alignment for both integers and floating-point numbers. Suppose further that we plan to use contiguous row-major layout for multidimensional arrays, that we do not wish to reorder fields of records or pack either records or arrays, and that we will assume without checking that all array subscripts are in bounds.

(a) Consider the following variable declarations.

```
var A : array [1..10, 10..100] of real;
    i : integer;
    x : real;
```

Show the code that our compiler should generate for the following assignment: `x := A[3,i]`. Explain how you arrived at your answer.

(b) Consider the following more complex declarations.

```
var r : record
        x : integer;
        y : char;
        A : array [1..10, 10..20] of record
                z : real;
                B : array [0..71] of char;
            end;
    end;
var j, k : integer;
```

Assume that these declarations are local to the current subroutine. Note the lower bounds on indices in `A`; the first element is `A[1,10]`.

Describe how `r` would be laid out in memory. Then show code to load `r.A[2,j].B[k]` into a register. Be sure to indicate which portions of the address calculation could be performed at compile time.

7.17 Suppose `A` is a 10×10 array of (4-byte) integers, indexed from [0][0] through [9][9]. Suppose further that the address of `A` is currently in register `r1`, the value of integer `i` is currently in register `r2`, and the value of integer `j` is currently in register `r3`.

Give pseudo-assembly language for a code sequence that will load the value of `A[i][j]` into register `r1` (a) assuming that `A` is implemented using (row-major) contiguous allocation; (b) assuming that `A` is implemented using row pointers. Each line of your pseudocode should correspond to a single instruction on a typical modern machine. You may use as many registers as you need. You need not preserve the values in `r1`, `r2`, and `r3`. You may assume that `i` and `j` are in bounds, and that addresses are 4 bytes long.

Which code sequence is likely to be faster? Why?

7.18 In Examples 7.62 and 7.63, show the code that would be required to access `A[i, j, k]` if subscript bounds checking were required.

7.19 Pointers and recursive type definitions complicate the algorithm for determining structural equivalence of types. Consider, for example, the following definitions:

```
type A = record
    x : pointer to B
    y : real
type B = record
    x : pointer to A
    y : real
```

The simple definition of structural equivalence given in Section 7.2.1 (expand the subparts recursively until all you have is a string of built-in types and type constructors; then compare them) does not work: we get an infinite expansion (type A = record x : pointer to record x : pointer to record x : pointer to record . . .). The obvious reinterpretation is to say two types A and B are equivalent if any sequence of field selections, array subscripts, pointer dereferences, and other operations that takes one down into the structure of A, and that ends at a built-in type, always ends at the same built-in type when used to dive into the structure of B (and encounters the same field names along the way). Under this reinterpretation, A and B above have the same type. Give an algorithm based on this reinterpretation that could be used in a compiler to determine structural equivalence. (Hint: the fastest approach is due to J. Král [Krá73]. It is based on the algorithm used to find the smallest deterministic finite automaton that accepts a given regular

language. This algorithm was outlined in Example 2.15 [page 59]; details can be found in any automata theory textbook [e.g., [HMU01]].)

7.20 Explain the meaning of the following C declarations:

```
double *a[n];
double (*b)[n];
double (*c[n])();
double (*d())[n];
```

7.21 In Ada 83, as in Pascal, pointers (`access` variables) can point only to objects in the heap. Ada 95 allows a new kind of pointer, the `access all` type, to point to other objects as well, provided that those objects have been declared to be `aliased`:

```
type int_ptr is access all Integer;
foo : aliased Integer;
ip : int_ptr;
...
ip := foo'Access;
```

The `'Access` attribute is roughly equivalent to C's "address of" (`&`) operator. How would you implement `access all` types and `aliased` objects? How would your implementation interact with automatic garbage collection (assuming it exists) for objects in the heap?

7.22 As noted in Section 7.7.2, Ada 95 forbids an `access all` pointer from referring to any object whose lifetime is briefer than that of the pointer's type. Can this rule be enforced completely at compile time? Why or why not?

7.23 In much of the discussion of pointers in Section 7.7, we assumed implicitly that every pointer into the heap points to the *beginning* of a dynamically allocated block of storage. In some languages, including Algol 68 and C, pointers may also point to data *inside* a block in the heap. If you were trying to implement dynamic semantic checks for dangling references or, alternatively, automatic garbage collection (precise or conservative), how would your task be complicated by the existence of such "internal pointers"?

7.24 (a) Occasionally one encounters the suggestion that a garbage-collected language should provide a `delete` operation as an optimization: by explicitly `delete`-ing objects that will never be used again, the programmer might save the garbage collector the trouble of finding and reclaiming those objects automatically, thereby improving performance. What do you think of this suggestion? Explain.

(b) Alternatively, one might allow the programmer to "tenure" an object, so that it will never be a candidate for reclamation. Is this a good idea?

7.25 In Example 7.88 we noted that functional languages can safely use reference counts since the lack of an assignment statement prevents them from introducing circularity. This isn't strictly true; constructs like the Lisp `letrec`

can also be used to make cycles, so long as uses of circularly defined names are hidden inside `lambda` expressions in each definition:

```
(define foo (lambda ()
    (letrec ((a (lambda(f) (if f #\A b)))
             (b (lambda(f) (if f #\B c)))
             (c (lambda(f) (if f #\C a))))
        a)))
```

Each of the functions `a`, `b`, and `c` contains a reference to the next:

```
((foo) #t)                  ⟹ #\A
(((foo) #f) #t)             ⟹ #\B
((((foo) #f) #f) #t)        ⟹ #\C
(((((foo) #f) #f) #f) #t)   ⟹ #\A
```

How might you address this circularity without giving up on reference counts?

7.26 Here is a skeleton for the standard quicksort algorithm in Haskell:

```
quicksort [] = []
quicksort (a : l) = quicksort [...] ++ [a] ++ quicksort [...]
```

The `++` operator denotes list concatenation (similar to `@` in ML). The `:` operator is equivalent to ML's `::` or Lisp's `cons`. Show how to express the two elided expressions as list comprehensions.

© **7.27–7.39** In More Depth.

7.13 Explorations

7.40 Some language definitions specify a particular representation for data types in memory, while others specify only the semantic behavior of those types. For languages in the latter class, some implementations guarantee a particular representation, while others reserve the right to choose different representations in different circumstances. Which approach do you prefer? Why?

7.41 If you have access to a compiler that provides optional dynamic semantic checks for out-of-bounds array subscripts, use of an inappropriate record variant, and/or dangling or uninitialized pointers, experiment with the cost of these checks. How much do they add to the execution time of programs that make a significant number of checked accesses? Experiment with different levels of optimization (code improvement) to see what effect each has on the overhead of checks.

7.42 Investigate the *typestate* mechanism employed by Strom et al. in the Hermes programming language [SBG+91]. Discuss its relationship to the notion of definite assignment in Java and C# (Section 6.1.3).

7.43 Investigate the notion of *type conformance*, employed by Black et al. in the Emerald programming language [BHJL07]. Discuss how conformance relates to the type inference of ML and to the class-based typing of object-oriented languages.

7.44 Write a library package that might be used by a language implementation to manage sets of elements drawn from a very large base type (e.g., `integer`). You should support membership tests, union, intersection, and difference. Does your package allocate memory from the heap? If so, what would a compiler that assumed the use of your package need to do to make sure that space was reclaimed when no longer needed?

7.45 Learn about SETL [SDDS86], a programming language based on sets, designed by Jack Schwartz of New York University. List the mechanisms provided as built-in set operations. Compare this list with the set facilities of other programming languages. What data structure(s) might a SETL implementation use to represent sets in a program?

7.46 The HotSpot Java compiler and virtual machine implements an entire suite of garbage collectors: a traditional generational collector, a compacting collector for the old generation, a low pause-time parallel collector for the nursery, a high-throughput parallel collector for the old generation, and a "mostly concurrent" collector for the old generation that runs in parallel with the main program. Learn more about these algorithms. When is each used, and why?

7.47 Implement your favorite garbage collection algorithm in Ada 95. Alternatively, implement a special pointer class in C++ for which storage is garbage collected. You'll want to use templates (generics) so that your class can be instantiated for arbitrary pointed-to types.

7.48 Experiment with the cost of garbage collection in your favorite language implementation. What kind of collector does it use? Can you create artificial programs for which it performs particularly well or poorly?

7.49 Learn about *weak references* in Java. How do they interact with garbage collection? Describe several scenarios in which they may be useful.

7.50–7.53 In More Depth.

7.14 Bibliographic Notes

References to general information on the various programming languages mentioned in this chapter can be found in Appendix A, and in the Bibliographic Notes for Chapters 1 and 6. Welsh, Sneeringer, and Hoare [WSH77] provide a critique of the original Pascal definition, with a particular emphasis on its type system.

Tanenbaum's comparison of Pascal and Algol 68 also focuses largely on types [Tan78]. Cleaveland [Cle86] provides a book-length study of many of the issues in this chapter. Pierce [Pie02] provides a formal and detailed modern coverage of the subject. The ACM Special Interest Group on Programming Languages launched a biennial workshop on *Types in Language Design and Implementation* in 2003.

What we have referred to as the denotational model of types originates with Hoare [DDH72]. Denotational formulations of the overall semantics of programming languages are discussed in the Bibliographic Notes for Chapter 4. A related but distinct body of work uses algebraic techniques to formalize data abstraction; key references include Guttag [Gut77] and Goguen et al. [GTW78]. Milner's original paper [Mil78] is the seminal reference on type inference in ML. Mairson [Mai90] proves that the cost of unifying ML types is $O(2^n)$, where n is the length of the program. Fortunately, the cost is linear in the size of the program's type expressions, so the worst case arises only in programs whose semantics are too complex for a human being to understand anyway.

Hoare [Hoa75] discusses the definition of recursive types under a reference model of variables. Cardelli and Wegner survey issues related to polymorphism, overloading, and abstraction [CW85]. The new Character Model standard for the World Wide Web provides a remarkably readable introduction to the subtleties and complexities of multilingual character sets [Wor05].

Tombstones are due to Lomet [Lom75, Lom85]. Locks and keys are due to Fischer and LeBlanc [FL80]. The latter also discuss how to check for various other dynamic semantic errors in Pascal, including those that arise with variant records. Constant-space (pointer-reversing) mark-and-sweep garbage collection is due to Schorr and Waite [SW67]. Stop-and-copy collection was developed by Fenichel and Yochelson [FY69], based on ideas due to Minsky. Deutsch and Bobrow [DB76] describe an *incremental* garbage collector that avoids the "stop-the-world" phenomenon. Wilson and Johnstone [WJ93] describe a later incremental collector. The conservative collector described at the end of Section 7.7.3 is due to Boehm and Weiser [BW88]. Cohen [Coh81] surveys garbage-collection techniques as of 1981; Wilson [Wil92b] and Jones and Lins [JL96] provide somewhat more recent views.